Diabetes Hospital Readmission

Data Analysis and Visualization: Certification Project Report

Name: Manil Bista

Date: 2083/06/26

Repository: https://github.com/ManilBista/healthcare-readmission-analysis

Dataset: UCI Diabetes 130-US Hospitals for Years 1999-2008

Objective

This project analyses the UCI Diabetes 130-US Hospitals dataset (1999-2008). Each row of the raw file is one hospital encounter of a patient with diabetes. The question is: which patients are readmitted within 30 days of discharge? The report follows the steps of the certification task and shows the notebook code and its saved output for each step.

How to read the figures. "Figure N" shows the code of a notebook cell with line numbers. "Figure N-out" shows the output saved in the notebook for that cell, and "Figure N-chart" shows a saved chart. Nothing was rerun. Cell numbers count from 0 in the notebook file. Outputs longer than 20 lines show the first 12 lines, "..." and the last 3 lines.

1. Data Collection and Exploration

Notebook Setup and Part 1. Part 1 only looks at the raw data (df_raw). Nothing is changed here.

1.1 Dataset loading

What was done. The Setup cell reads diabetic_data.csv into the pandas DataFrame df_raw with pd.read_csv. IDS_mapping.csv holds three lookup tables stacked in one file, so it is read once as a whole (mapping) and once per table with nrows and skiprows. **Why:** every later part reuses these tables, so they are loaded once. **Result:** this cell has no saved output. The shape is shown in Figure 4-out.

Cell 4

```
1  # main dataset: one row per hospital encounter (raw, never changed)
2  df_raw = pd.read_csv(f"{DATA_DIR}/diabetic_data.csv")
3
4  # the mapping file as one stacked table (only to look at it in Part 1)
5  mapping = pd.read_csv(f"{DATA_DIR}/IDS_mapping.csv")
6
7  # the same file split into its 3 lookup tables
8  admission_type_map = pd.read_csv(f"{DATA_DIR}/IDS_mapping.csv", nrows=8)
9  discharge_map = pd.read_csv(f"{DATA_DIR}/IDS_mapping.csv", skiprows=10, nrows=30)
10 admission_source_map = pd.read_csv(f"{DATA_DIR}/IDS_mapping.csv", skiprows=42)
11
12 print("diabetic_data:", df_raw.shape)
13 print("lookup tables:", admission_type_map.shape, discharge_map.shape, admission_source_map.shape)
```

Figure 1: Load diabetic_data.csv and the three IDS_mapping.csv lookup tables with pandas

The first five rows show the raw values. The weight column already shows the "?" placeholder.

Cell 7

```
1  df_raw.head()
```

Figure 2: First look at the raw data with df_raw.head()

```
Output of cell 7
```

```
   encounter_id  patient_nbr             race  gender      age  weight  \
0       2278392      8222157        Caucasian  Female   [0-10)       ?  \
1        149190     55629189        Caucasian  Female  [10-20)       ?  \
2         64410     86047875  AfricanAmerican  Female  [20-30)       ?  \
3        500364     82442376        Caucasian    Male  [30-40)       ?  \
4         16680     42519267        Caucasian    Male  [40-50)       ?  \

   admission_type_id  discharge_disposition_id  admission_source_id  \
0                  6                        25                    1  \
1                  1                         1                    7  \
2                  1                         1                    7  \
3                  1                         1                    7  \
4                  1                         1                    7  \

   time_in_hospital  ...
0                 1  ...
1                 3  ...
2                 2  ...
3                 2  ...
4                 1  ...

[5 rows x 50 columns]
```

Figure 2-out: output (cell 7)

The mapping file has 67 rows and 2 columns. The three tables sit on top of each other: row 9 is the header of the discharge table.

```
Cell 39
```

```
1 # the mapping file read as one table just to look at it (loaded in Setup)
2 print(mapping.shape)
3 mapping.head(12)
```

Figure 3: Look at the stacked ID mapping file

```
Output of cell 39
```

```
(67, 2)
          admission_type_id  \
0                         1
1                         2
2                         3
3                         4
4                         5
5                         6
6                         7
7                         8
8                       NaN
9   discharge_disposition_id
...
                                      description
9                                     description
10                              Discharged to home
11  Discharged/transferred to another short term h...
```

Figure 3-out: output (cell 39)

1.2 Structure: rows, columns and data types

The raw data has **101,766 rows and 50 columns**. 13 columns are numbers (int64) and 37 are text (object). Some numbers are really codes: admission_type_id, discharge_disposition_id and admission_source_id only make sense with the mapping file.

Cell 9

```
1 df_raw.shape   # (rows, columns)
```

Figure 4: Number of rows and columns

Output of cell 9

```
(101766, 50)
```

Figure 4-out: output (cell 9)

Cell 10

```
1 df_raw.info()   # data type + non-null count of every column
```

Figure 5: Data type and non-null count of every column

Output of cell 10

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101766 entries, 0 to 101765
Data columns (total 50 columns):
 #   Column                    Non-Null Count   Dtype
---  ------                    --------------   -----
 0   encounter_id              101766 non-null  int64
 1   patient_nbr               101766 non-null  int64
 2   race                      101766 non-null  object
 3   gender                    101766 non-null  object
 4   age                       101766 non-null  object
 5   weight                    101766 non-null  object
 6   admission_type_id         101766 non-null  int64
...
 49  readmitted                101766 non-null  object
dtypes: int64(13), object(37)
memory usage: 38.8+ MB
```

Figure 5-out: output (cell 10)

Cell 11

```
1 # how many columns of each data type
2 df_raw.dtypes.value_counts()
```

Figure 6: Count of columns per data type

Output of cell 11

```
object    37
int64     13
Name: count, dtype: int64
```

Figure 6-out: output (cell 11)

1.3 Summary statistics

What was done: describe() gives count, mean, std, min, quartiles and max of the numeric columns. describe(include="object") gives count, number of unique values, most common value and its frequency for the text columns. **What it shows:**

- encounter_id and patient_nbr are IDs, so their mean and std mean nothing.
- number_emergency has a 75% value of 0 and a max of 76. number_inpatient has a 75% value of 1 and a max of 21. A few extreme values exist.
- citoglipton has only 1 unique value (examide too, in the hidden part of the output).
- diag_1, diag_2 and diag_3 have 717, 749 and 790 different codes.

Cell 24

```
1 # numeric columns
2 df_raw.describe()
```

Figure 7: Summary statistics of the numeric columns

Output of cell 24

```
       encounter_id   patient_nbr  admission_type_id  \
count  1.017660e+05  1.017660e+05      101766.000000
mean   1.652016e+08  5.433040e+07           2.024006
std    1.026403e+08  3.869636e+07           1.445403
min    1.252200e+04  1.350000e+02           1.000000
25%    8.496119e+07  2.341322e+07           1.000000
50%    1.523890e+08  4.550514e+07           1.000000
75%    2.302709e+08  8.754595e+07           3.000000
max    4.438672e+08  1.895026e+08           8.000000

       discharge_disposition_id  admission_source_id  time_in_hospital  \
count             101766.000000        101766.000000     101766.000000
...
       number_emergency  number_inpatient  number_diagnoses
50%            0.000000          0.000000          8.000000
75%            0.000000          1.000000          9.000000
max           76.000000         21.000000         16.000000
```

Figure 7-out: output (cell 24)

Cell 25

```
1 # text columns: count, number of unique values, most common value and its frequency
2 df_raw.describe(include="object")
```

Figure 8: Summary of the text columns

Output of cell 25

```
            race  gender      age  weight payer_code medical_specialty  \
count      101766  101766   101766  101766     101766            101766
unique          6       3       10      10         18                73
top     Caucasian  Female  [70-80)       ?          ?                 ?
freq        76099   54708    26068   98569      40256             49949

        diag_1  diag_2  diag_3 max_glu_serum  ... citoglipton insulin  \
count   101766  101766  101766          5346  ...      101766  101766
unique     717     749     790             3  ...           1       4
top        428     276     250          Norm  ...          No      No
freq      6862    6752   11555          2597  ...      101766   47383

...
       readmitted
freq        54864

[4 rows x 37 columns]
```

Figure 8-out: output (cell 25)

1.4 Problems found

"?" placeholders: The file writes "?" for unknown values. pandas does not treat "?" as missing, so it was counted directly. 7 columns have it: weight 98,569 rows (96.86%), medical_specialty 49,949 (49.08%), payer_code 40,256 (39.56%), race 2,273 (2.23%), diag_3 1,423, diag_2 358 and diag_1 21.

Cell 14

```python
# count "?" in every column
question_marks = (df_raw == "?").sum()

# keep only the columns that have at least one "?"
question_marks[question_marks > 0].sort_values(ascending=False)
```

Figure 9: Count "?" placeholders per column

Output of cell 14

```
weight              98569
medical_specialty   49949
payer_code          40256
race                 2273
diag_3               1423
diag_2                358
diag_1                 21
dtype: int64
```

Figure 9-out: output (cell 14)

Cell 15

```python
# same thing as a percentage of all rows
question_pct = ((df_raw == "?").mean() * 100).round(2)
question_pct[question_pct > 0].sort_values(ascending=False)
```

Figure 10: "?" placeholders as a percentage of rows

```
Output of cell 15
weight               96.86
medical_specialty    49.08
payer_code           39.56
race                  2.23
diag_3                1.40
diag_2                0.35
diag_1                0.02
dtype: float64
```

Figure 10-out: output (cell 15)

Missing values: max_glu_serum and A1Cresult hold the text None, which means the test was not done. pandas reads it as NaN: 96,420 and 84,748 rows. With "?" counted as missing too, weight (96.86%) and max_glu_serum (94.75%) are above 90%. A1Cresult is at 83.28%.

Cell 17

```python
# NaN count per column (only columns that have some)
nan_count = df_raw.isna().sum()
nan_count[nan_count > 0]
```

Figure 11: Count real NaN values per column

```
Output of cell 17
max_glu_serum    96420
A1Cresult        84748
dtype: int64
```

Figure 11-out: output (cell 17)

Cell 19

```python
# a temporary copy where "?" is also treated as missing (only for counting, df_raw is not changed)
df_check = df_raw.replace("?", np.nan)

# total missing % per column, worst first
missing_pct = (df_check.isna().mean() * 100).round(2)
missing_pct = missing_pct[missing_pct > 0].sort_values(ascending=False)
missing_pct
```

Figure 12: Total missing % per column (NaN and "?")

```
Output of cell 19
weight               96.86
max_glu_serum        94.75
A1Cresult            83.28
medical_specialty    49.08
payer_code           39.56
race                  2.23
diag_3                1.40
diag_2                0.35
diag_1                0.02
dtype: float64
```

Figure 12-out: output (cell 19)

Cell 20

```
# bar chart of missing % per column
plt.figure(figsize=(9, 4))
plt.bar(missing_pct.index, missing_pct.values)
plt.axhline(90, color="red", linestyle="--", label="90% line")
plt.title("Missing Values per Column (NaN + '?')")
plt.xlabel("Column")
plt.ylabel("Missing (%)")
plt.xticks(rotation=45)
plt.legend()
plt.show()
```

Figure 13: Bar chart of missing % per column

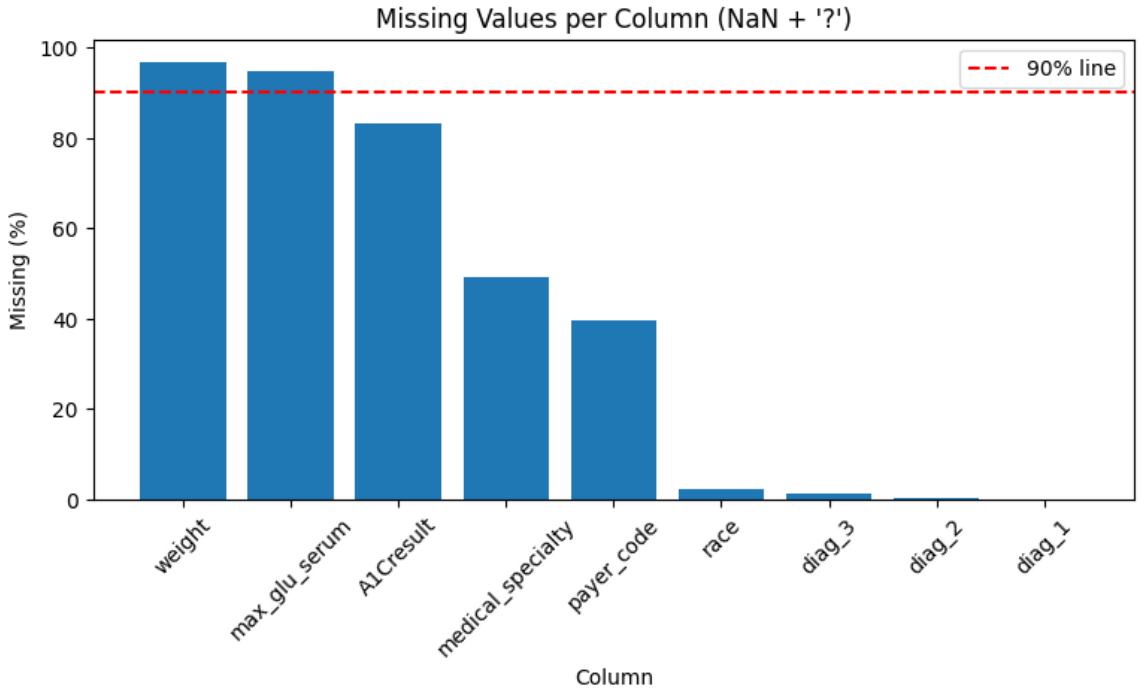


Figure 13-chart: output chart, Missing Values per Column (NaN + '?') (cell 20)

Constant columns: examide and citoglipton have one value (No) in every row (Figure 8-out). They cannot explain readmission.

Invalid gender: 3 rows have gender Unknown/Invalid.

Cell 21

```
# gender has its own placeholder value
df_raw["gender"].value_counts()
```

Figure 14: Value counts of gender

```
Output of cell 21
gender
Female             54708
Male               47055
Unknown/Invalid        3
Name: count, dtype: int64
```

Figure 14-out: output (cell 21)

Repeated patients: The 101,766 encounters belong to only 71,518 patients. One patient has up to 40 encounters. There are no fully duplicated rows. Repeat visits of the same patient are not independent rows.

```
Cell 28
1 print("Rows (encounters):", len(df_raw))
2 print("Unique encounter_id:", df_raw["encounter_id"].nunique())
3 print("Unique patients:", df_raw["patient_nbr"].nunique())
4 print("Fully duplicated rows:", df_raw.duplicated().sum())
```

Figure 15: Encounters, unique patients and duplicated rows

```
Output of cell 28
Rows (encounters): 101766
Unique encounter_id: 101766
Unique patients: 71518
Fully duplicated rows: 0
```

Figure 15-out: output (cell 28)

```
Cell 29
1 # how many encounters each patient has
2 visits_per_patient = df_raw["patient_nbr"].value_counts()
3 visits_per_patient.describe()
```

Figure 16: Encounters per patient

```
Output of cell 29
count    71518.000000
mean         1.422942
std          1.090740
min          1.000000
25%          1.000000
50%          1.000000
75%          1.000000
max         40.000000
Name: count, dtype: float64
```

Figure 16-out: output (cell 29)

Outliers: Count columns such as number_emergency (max 76) and number_inpatient (max 21) sit far above their 75% value (Figure 7-out). They are checked with the IQR rule in Part 2, Step 9.

Class imbalance: readmitted has 3 classes: NO 54,864 (53.91%), >30 35,545 (34.93%) and <30 11,357 (11.16%). The question is about <30, so a binary target readmitted_30 was built: 11,357 encounters (11.16%) against 90,409 (88.84%). A model that always guesses "not readmitted" would be right 88.84% of the time and still find no readmitted patient, so accuracy alone is not a useful measure.

Cell 32

```
# the 3 original classes
df_raw["readmitted"].value_counts()
```

Cell 33

```
# same as percentage
(df_raw["readmitted"].value_counts(normalize=True) * 100).round(2)
```

Figure 17: The three original readmitted classes, count and %

Output of cell 32

```
readmitted
NO     54864
>30    35545
<30    11357
Name: count, dtype: int64
```

Output of cell 33

```
readmitted
NO     53.91
>30    34.93
<30    11.16
Name: proportion, dtype: float64
```

Figure 17-out: output (cells 32, 33)

Cell 34

```
# binary target: 1 = readmitted within 30 days, 0 = the rest (NO or >30)
df_raw["readmitted_30"] = (df_raw["readmitted"] == "<30").astype(int)

df_raw["readmitted_30"].value_counts()
```

Cell 35

```
# percentage of each class
(df_raw["readmitted_30"].value_counts(normalize=True) * 100).round(2)
```

Figure 18: Binary target readmitted_30, count and %

Output of cell 34

```
readmitted_30
0    90409
1    11357
Name: count, dtype: int64
```

Output of cell 35

```
readmitted_30
0    88.84
1    11.16
Name: proportion, dtype: float64
```

Figure 18-out: output (cells 34, 35)

Cell 36

```
# bar chart of the binary target
counts = df_raw["readmitted_30"].value_counts().sort_index()

plt.figure(figsize=(6, 4))
plt.bar(["Not <30 days (0)", "<30 days (1)"], counts.values, color=["steelblue", "tomato"])
plt.title("Class Balance: Readmitted Within 30 Days")
plt.xlabel("Readmission class")
plt.ylabel("Number of encounters")
plt.show()
```

Figure 19: Bar chart of the binary target

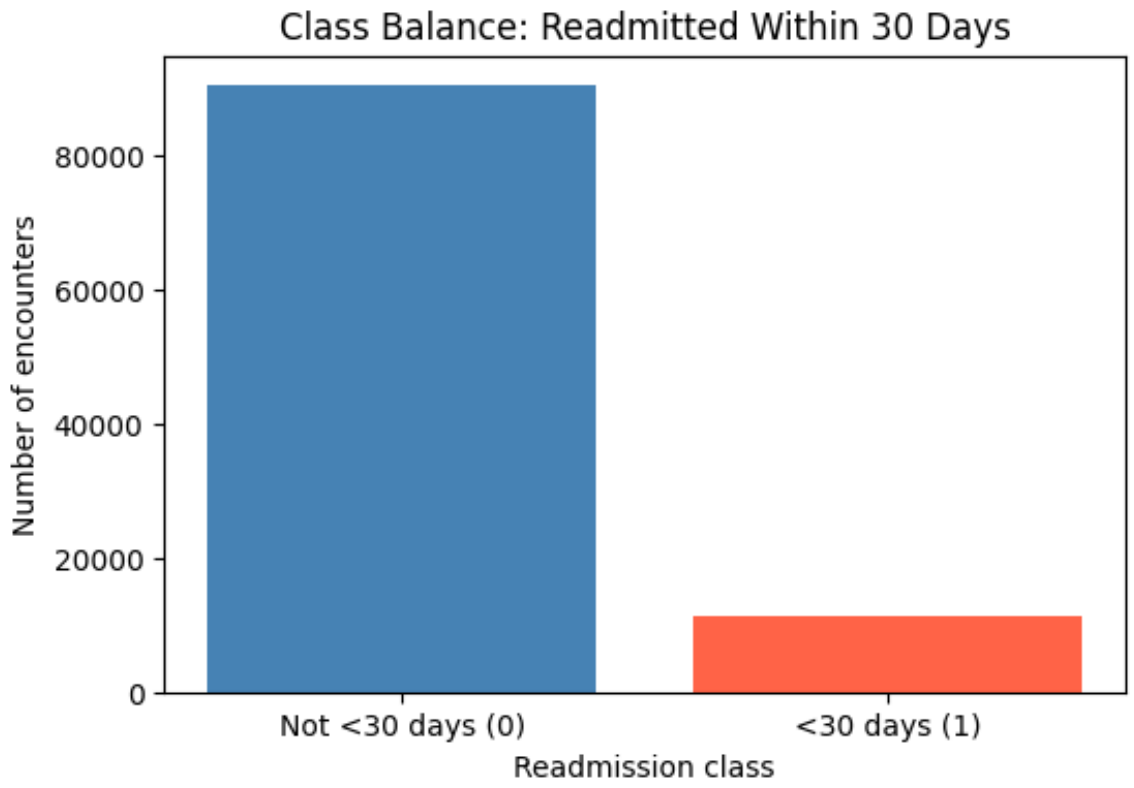


Figure 19-chart: output chart, Class Balance: Readmitted Within 30 Days (cell 36)

These are encounter counts. After cleaning (one encounter per patient) the target rate is 8.97% (Part 2, Step 13).

Summary of problems and where they are treated:

Problem	Evidence (saved output)	Treated in Part 2
"?" placeholders	7 columns, weight 96.86%	Step 2
Columns over 90% missing	weight 96.86%, max_glu_serum 94.75%	Step 3
Constant columns	examide, citoglipton: 1 unique value	Step 4
Other missing values	A1Cresult 83.28%, medical_specialty 49.08%, payer_code 39.56%, race 2.23%	Step 5
Repeated patients	101,766 encounters, 71,518 patients	Step 6
Invalid gender	3 rows Unknown/Invalid	Step 8
Outliers	number_emergency max 76 with 75% = 0	Step 9
Class imbalance	<30: 11.16% of encounters	Step 13 (target); Part 4 (class weights)

2. Data Cleaning and Transformation

Notebook Part 2. Input: diabetic_data.csv (101,766 rows x 50 columns). The cleaning produces two files:

Output file	Contents
data/cleaned_readable.csv	69,970 rows x 51 columns. Real units and text labels, for EDA and Power BI.
data/cleaned.csv	69,970 rows x 140 columns. Encoded and scaled, for modeling.

The table below is the cleaning log from the existing report. Each step after it shows the code, the saved output and the decisions it covers.

#	Decision	Reason	Effect
1	Replace "?" with NaN	The file uses "?" for unknown values, which pandas does not detect as missing	7 columns had "?"; 0 left after replacing
2	Drop columns with more than 90% missing	Too little data to be useful; filling 90%+ of a column would be mostly guessing	Dropped weight (96.86% missing) and max_glu_serum (94.75%). 50 -> 48 columns
3	Drop columns with only one value	A constant column cannot explain readmission	Dropped examide and citoglipton (every row = No). 48 -> 46 columns
4	Fill A1Cresult NaN with "Not tested"	Missing here means the HbA1c test was not done, which is real information	83.28% of raw rows
5	Fill medical_specialty, payer_code, race NaN with "Unknown"	Too many missing to drop rows (49.08% and 39.56%), and race should never be guessed	Kept as their own category
6	Keep one encounter per patient (the first, by smallest encounter_id)	Repeat visits of the same patient are not independent rows and could leak information between train and	101,766 -> 71,518 rows

#	Decision	Reason	Effect
		test sets. Same choice as the original study (Strack et al. 2014)	
7	Remove discharges coded Expired / Hospice (IDs 11, 13, 14, 19, 20, 21)	These patients cannot be readmitted, so they would bias the model toward "not readmitted"	1,545 rows removed. 71,518 -> 69,973
8	Remove gender = Unknown/Invalid	Invalid placeholder value, only 3 rows	69,973 -> 69,970 rows
9	Cap outliers at Q3 + 1.5 * IQR (upper side only)	Capping keeps every patient but limits the pull of extreme values. Lower limits are 0 or below (nothing flagged), except number_diagnoses (1.5), where a value of 1 is a valid patient	time_in_hospital capped at 12 (1,403 rows), num_lab_procedures at 96 (98), num_procedures at 5 (3,845), num_medications at 35 (1,867), number_diagnoses at 13.5 (42)
10	Do not cap number_outpatient, number_emergency, number_inpatient	Their Q1 = Q3 = 0, so IQR = 0 and the upper limit = 0. Capping would erase every past visit (9,119 / 5,100 / 8,191 rows), the most useful information	Left unchanged
11	Group ICD-9 codes of diag_1/2/3 into 9 categories (Circulatory, Respiratory, Digestive, Diabetes, Injury, Musculoskeletal, Genitourinary, Neoplasms, Other)	~700-800 raw codes per column are too many categories. ICD-9 chapter grouping from Strack et al. 2014. V/E codes and the few missing codes go to Other	New diag_1_group, diag_2_group, diag_3_group; raw columns dropped. Largest primary group: Circulatory (21,383 patients)

#	Decision	Reason	Effect
12	Keep the 10 most common medical specialties, put the rest in "Other"	71 specialties, most of them rare. One-hot encoding all of them would add many almost-empty columns	71 -> 12 values (10 + Other + Unknown)
13	New feature total_visits = outpatient + emergency + inpatient	One measure of how much hospital care was needed in the past year	mean 0.56, max 49
14	New feature age_mid = middle of the age band ([70-80) -> 75)	Turns the text band into a number that can be scaled and tested	mean 65.44
15	New feature num_active_meds = number of the 21 medicines not equal to No	How complex the diabetes treatment is	mean 1.19, max 6
16	New feature num_med_changes = number of medicines with Up or Down	How much the treatment was adjusted during the stay	mean 0.26, max 4
17	Target readmitted_30 = 1 if readmitted == "<30", else 0	The project question is early (30-day) readmission	6,277 positives (8.97%) vs 63,693 (91.03%)
18	Save cleaned_readable.csv before encoding and scaling	EDA charts and Power BI need real units (days, years) and text labels	69,970 x 51
19	Label-encode gender (Male = 1), change (Ch = 1), diabetesMed (Yes = 1)	Only two values each	0 / 1 columns
20	Encode the 21 medicine columns as 0 = No, 1 =	One-hot on 21 columns x 4 values would add ~60 mostly-	0 / 1 columns

#	Decision	Reason	Effect
	taken (Steady / Up / Down)	empty columns. The Up / Down direction is already kept in num_med_changes	
21	One-hot encode race, payer_code, medical_specialty, A1Cresult, the 3 diagnosis groups, admission_type_id, discharge_disposition_id, admission_source_id (drop_first=True)	Categories with no natural order. The ID columns are codes, so they are converted to text first. drop_first removes one redundant column per group	140 columns after encoding
22	Drop encounter_id, patient_nbr, age (text), readmitted (text) from the model file	IDs carry no medical meaning; age and readmitted were replaced by age_mid and readmitted_30	Not in cleaned.csv (still in cleaned_readable.csv)
23	Standard-scale 12 numeric columns (mean 0, std 1)	Columns have very different ranges, and logistic regression is sensitive to scale. 0/1 columns and the target are not scaled	Verified mean = 0.0, std = 1.0

2.1 Missing values

This part handles the placeholders, missing values and the rows that should not be in the analysis.

Step 1: Start from the raw data

Done: Part 2 works on a copy of df_raw without the exploration target. **Why:** the raw table stays unchanged. **Result:** 101,766 rows x 50 columns.

Cell 45

```
# work on a copy; the exploration target column is rebuilt in Step 13
df = df_raw.drop(columns=["readmitted_30"]).copy()

print("Starting shape:", df.shape)
```

Figure 20: Start from a copy of the raw data

```
Output of cell 45
Starting shape: (101766, 50)
```

Figure 20-out: output (cell 45)

Step 2: Replace "?" with NaN (decision 1)

Done: every "?" became NaN. **Why:** pandas does not see "?" as missing. **Result:** 0 "?" left.

```
Cell 47
1 df = df.replace("?", np.nan)
2
3 # check: no "?" left anywhere
4 print('"?" left:', (df == "?").sum().sum())
```

Figure 21: Replace "?" with NaN

```
Output of cell 47
"?" left: 0
```

Figure 21-out: output (cell 47)

Step 3: Drop columns with more than 90% missing (decision 2)

Done: columns above 90% missing were dropped. **Why:** filling 90% or more of a column would be mostly guessing. **Result:** weight (96.86%) and max_glu_serum (94.75%) dropped, 50 -> 48 columns.

```
Cell 49
1 # missing % per column
2 missing_pct = (df.isna().mean() * 100).round(2)
3 missing_pct[missing_pct > 0].sort_values(ascending=False)
Cell 50
1 # columns above the 90% line
2 cols_over_90 = missing_pct[missing_pct > 90].index.tolist()
3 print("Dropping:", cols_over_90)
4
5 df = df.drop(columns=cols_over_90)
6 print("Shape now:", df.shape)
```

Figure 22: Drop columns with more than 90% missing

Output of cell 49

```
weight               96.86
max_glu_serum        94.75
A1Cresult            83.28
medical_specialty    49.08
payer_code           39.56
race                  2.23
diag_3                1.40
diag_2                0.35
diag_1                0.02
dtype: float64
```

Output of cell 50

```
Dropping: ['weight', 'max_glu_serum']
Shape now: (101766, 48)
```

Figure 22-out: output (cells 49, 50)

Step 4: Drop columns with only one value (decision 3)

Done: examide and citoglipton were dropped. **Why:** a constant column cannot explain readmission. **Result:** 48 -> 46 columns.

Cell 53

```
# number of unique values per column, smallest first
df.nunique().sort_values().head(5)
```

Cell 54

```
single_value_cols = df.columns[df.nunique() == 1].tolist()
print("Dropping:", single_value_cols)

df = df.drop(columns=single_value_cols)
print("Shape now:", df.shape)
```

Figure 23: Drop columns with only one value

Output of cell 53

```
examide          1
citoglipton      1
acetohexamide    2
tolbutamide      2
diabetesMed      2
dtype: int64
```

Output of cell 54

```
Dropping: ['examide', 'citoglipton']
Shape now: (101766, 46)
```

Figure 23-out: output (cells 53, 54)

Step 5: Fill the remaining missing values (decisions 4 and 5)

Done: A1Cresult NaN -> "Not tested". medical_specialty, payer_code and race NaN -> "Unknown". **Why:** a missing HbA1c result means the test was not done, which is real

information. medical_specialty (49.08%) and payer_code (39.56%) have too many missing values to drop rows, and race should never be guessed.

Result: only diag_1 (21), diag_2 (358) and diag_3 (1,423) still have missing values. They become "Other" in Step 10.

Cell 56

```
df["A1Cresult"] = df["A1Cresult"].fillna("Not tested")
df["medical_specialty"] = df["medical_specialty"].fillna("Unknown")
df["payer_code"] = df["payer_code"].fillna("Unknown")
df["race"] = df["race"].fillna("Unknown")

# what is still missing (only the diagnosis columns should be left)
df.isna().sum()[df.isna().sum() > 0]
```

Figure 24: Fill the remaining missing values

Output of cell 56

```
diag_1      21
diag_2     358
diag_3    1423
dtype: int64
```

Figure 24-out: output (cell 56)

Step 6: Keep one encounter per patient (decision 6)

Done: rows were sorted by encounter_id and the first encounter of each patient was kept. **Why:** repeat visits of the same patient are not independent and could leak information between train and test sets. This is the same choice as the original study (Strack et al. 2014). **Result:** 101,766 -> 71,518 rows.

Cell 58

```
print("Rows before:", len(df))
print("Unique patients:", df["patient_nbr"].nunique())

# sort so the earliest encounter comes first, then keep the first row of each patient
df = df.sort_values("encounter_id")
df = df.drop_duplicates(subset="patient_nbr", keep="first")

print("Rows after:", len(df))
```

Figure 25: Keep the first encounter of each patient

Output of cell 58

```
Rows before: 101766
Unique patients: 71518
Rows after: 71518
```

Figure 25-out: output (cell 58)

Step 7: Remove expired and hospice discharges (decision 7)

Done: discharge IDs 11, 13, 14, 19, 20 and 21 (Expired or Hospice) were removed. **Why:** these patients cannot be readmitted, so they would bias the model toward "not readmitted". **Result:** 1,545 rows removed, 71,518 -> 69,973.

Cell 60

```
# the discharge lookup table was loaded in Setup (discharge_map)
expired_hospice_ids = [11, 13, 14, 19, 20, 21]
discharge_map.loc[discharge_map["discharge_disposition_id"].isin(expired_hospice_ids)]
```

Cell 61

```
print("Rows to remove:", df["discharge_disposition_id"].isin(expired_hospice_ids).sum())

# keep only rows whose discharge code is NOT in the list (~ means NOT)
df = df.loc[~df["discharge_disposition_id"].isin(expired_hospice_ids)]

print("Rows after:", len(df))
```

Figure 26: Remove expired and hospice discharges

Output of cell 60

```
    discharge_disposition_id  \
10                        11
12                        13
13                        14
18                        19
19                        20
20                        21

                                        description
10                                          Expired
12                                   Hospice / home
13                        Hospice / medical facility
18          Expired at home. Medicaid only, hospice.
19  Expired in a medical facility. Medicaid only, ...
20    Expired, place unknown. Medicaid only, hospice.
```

Output of cell 61

```
Rows to remove: 1545
Rows after: 69973
```

Figure 26-out: output (cells 60, 61)

Step 8: Remove invalid gender rows (decision 8)

Done: rows with gender Unknown/Invalid were removed. **Why:** it is a placeholder, not a value. **Result:** 3 rows removed, 69,973 -> 69,970.

Cell 63

```
print(df["gender"].value_counts())

df = df.loc[df["gender"] != "Unknown/Invalid"]

print("Rows after:", len(df))
```

Figure 27: Remove invalid gender rows

```
Output of cell 63

gender
Female             37229
Male               32741
Unknown/Invalid        3
Name: count, dtype: int64
Rows after: 69970
```

Figure 27-out: output (cell 63)

Row count flow

101,766 → 71,518 → 69,973 → 69,970

Stage	Rows	Step
Raw data	101,766	Part 1
One encounter per patient	71,518	Step 6
Expired / hospice removed	69,973	Step 7
Invalid gender removed	69,970	Step 8

2.2 Outlier detection and treatment

Step 9: Treat outliers with IQR (decisions 9 and 10)

Method: the IQR rule. A value above Q3 + 1.5 x IQR is an outlier. Outliers are capped at that upper limit instead of deleted, so every patient is kept.

- **Upper side only:** The lower limits are 0 or below (nothing flagged), except number_diagnoses (1.5), where a value of 1 is a valid patient.
- **Three columns skipped:** number_outpatient, number_emergency and number_inpatient have Q1 = Q3 = 0, so IQR = 0 and the upper limit is 0. Capping would erase every past visit (9,119 / 5,100 / 8,191 rows), the most useful information.

Column	Q1	Q3	Upper limit	Rows above	Action
time_in_hospital	2	6	12	1,403	Capped at 12
num_lab_procedures	31	57	96	98	Capped at 96
num_procedures	0	2	5	3,845	Capped at 5
num_medications	10	20	35	1,867	Capped at 35
number_diagnoses	6	9	13.5	42	Capped at 13.5
number_outpatient	0	0	0	9,119	Skipped (IQR = 0)
number_emergency	0	0	0	5,100	Skipped (IQR = 0)

Column	Q1	Q3	Upper limit	Rows above	Action
number_inpatient	0	0	0	8,191	Skipped (IQR = 0)

Result: the new maximums in the last output match the caps (12, 96, 5, 35 and 13.5).

Cell 65

```
numeric_cols = ["time_in_hospital", "num_lab_procedures", "num_procedures", "num_medications",
                "number_outpatient", "number_emergency", "number_inpatient", "number_diagnoses"]

# look at the IQR limits of each column before changing anything
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    upper = Q3 + 1.5 * IQR
    print(col, "| Q1:", Q1, "| Q3:", Q3, "| IQR:", IQR, "| upper:", upper, "| above upper:", (df[col] > upper).sum())
```

Cell 66

```
# make the columns float first, because an upper limit like 13.5 is not a whole number
df[numeric_cols] = df[numeric_cols].astype(float)

# cap only the columns where IQR is bigger than 0
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    upper = Q3 + 1.5 * IQR

    if IQR == 0:
        print("Skipped (IQR = 0):", col)
    else:
        n_capped = (df[col] > upper).sum()
        df.loc[df[col] > upper, col] = upper
        print("Capped", col, "at", upper, "->", n_capped, "rows")
```

Cell 67

```
# check the new maximums
df[numeric_cols].describe().loc[["min", "max"]]
```

Figure 28: IQR limits, capping and new maximums

Output of cell 65

```
time_in_hospital | Q1: 2.0 | Q3: 6.0 | IQR: 4.0 | upper: 12.0 | above upper: 1403
num_lab_procedures | Q1: 31.0 | Q3: 57.0 | IQR: 26.0 | upper: 96.0 | above upper: 98
num_procedures | Q1: 0.0 | Q3: 2.0 | IQR: 2.0 | upper: 5.0 | above upper: 3845
num_medications | Q1: 10.0 | Q3: 20.0 | IQR: 10.0 | upper: 35.0 | above upper: 1867
number_outpatient | Q1: 0.0 | Q3: 0.0 | IQR: 0.0 | upper: 0.0 | above upper: 9119
number_emergency | Q1: 0.0 | Q3: 0.0 | IQR: 0.0 | upper: 0.0 | above upper: 5100
number_inpatient | Q1: 0.0 | Q3: 0.0 | IQR: 0.0 | upper: 0.0 | above upper: 8191
number_diagnoses | Q1: 6.0 | Q3: 9.0 | IQR: 3.0 | upper: 13.5 | above upper: 42
```

Output of cell 66

```
Capped time_in_hospital at 12.0 -> 1403 rows
Capped num_lab_procedures at 96.0 -> 98 rows
Capped num_procedures at 5.0 -> 3845 rows
Capped num_medications at 35.0 -> 1867 rows
Skipped (IQR = 0): number_outpatient
Skipped (IQR = 0): number_emergency
Skipped (IQR = 0): number_inpatient
Capped number_diagnoses at 13.5 -> 42 rows
```

Output of cell 67

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	\
min	1.0	1.0	0.0	1.0	
max	12.0	96.0	5.0	35.0	

	number_outpatient	number_emergency	number_inpatient	number_diagnoses
min	0.0	0.0	0.0	1.0
max	42.0	42.0	12.0	13.5

Figure 28-out: output (cells 65, 66, 67)

2.3 Data transformation: encoding, scaling and new features

Step 10: Group ICD-9 codes into diagnosis groups (decision 11)

Done: the function icd9_group maps each code of diag_1, diag_2 and diag_3 to one of 9 groups (Circulatory, Respiratory, Digestive, Diabetes, Injury, Musculoskeletal, Genitourinary, Neoplasms, Other). **Why:** about 700-800 raw codes per column are too many categories. The grouping follows the ICD-9 chapters used by Strack et al. 2014. V/E codes and the few missing codes go to Other. **Result:** new columns diag_1_group, diag_2_group, diag_3_group; the raw

columns were dropped. The largest primary group is Circulatory (21,383 patients). Cell 69 only defines the function and has no output.

Cell 69

```python
# turn one ICD-9 code into its group name
def icd9_group(code):
    # missing code
    if pd.isna(code):
        return "Other"
    # V and E codes are supplementary codes, not diseases
    if code.startswith("V") or code.startswith("E"):
        return "Other"
    # diabetes codes are 250.xx
    if code.startswith("250"):
        return "Diabetes"

    num = float(code)
    if (390 <= num < 460) or int(num) == 785:
        return "Circulatory"
    if (460 <= num < 520) or int(num) == 786:
        return "Respiratory"
    if (520 <= num < 580) or int(num) == 787:
        return "Digestive"
    if 800 <= num < 1000:
        return "Injury"
    if 710 <= num < 740:
        return "Musculoskeletal"
    if (580 <= num < 630) or int(num) == 788:
        return "Genitourinary"
    if 140 <= num < 240:
        return "Neoplasms"
    return "Other"
```

Figure 29 (part 1 of 2): Group ICD-9 codes into 9 diagnosis groups

Cell 70

```python
# apply the function to the 3 diagnosis columns -> 3 new group columns
df["diag_1_group"] = df["diag_1"].apply(icd9_group)
df["diag_2_group"] = df["diag_2"].apply(icd9_group)
df["diag_3_group"] = df["diag_3"].apply(icd9_group)

# the raw codes are no longer needed
df = df.drop(columns=["diag_1", "diag_2", "diag_3"])

df["diag_1_group"].value_counts()
```

Figure 29 (part 2 of 2): Group ICD-9 codes into 9 diagnosis groups

```
Output of cell 70
diag_1_group
Circulatory        21383
Other              12132
Respiratory         9486
Digestive           6487
Diabetes            5748
Injury              4692
Musculoskeletal     4064
Genitourinary       3440
Neoplasms           2538
Name: count, dtype: int64
```

Figure 29-out: output (cells 69, 70)

Step 11: Group rare medical specialties (decision 12)

Done: the 10 most common specialties were kept and the rest became "Other". **Why:** most of the 71 specialties are rare, and one-hot encoding all of them would add many almost empty columns. **Result:** 71 -> 12 values (10 + Other + Unknown).

```
Cell 72
print("Unique specialties before:", df["medical_specialty"].nunique())

# the 10 most common real specialties
top_10 = df.loc[df["medical_specialty"] != "Unknown", "medical_specialty"].value_counts().head(10).index.tolist()
print("Kept:", top_10)

# everything not in the top 10 and not Unknown becomes Other
df.loc[~df["medical_specialty"].isin(top_10 + ["Unknown"]), "medical_specialty"] = "Other"

print("Unique specialties after:", df["medical_specialty"].nunique())
```

Figure 30: Group rare medical specialties into Other

```
Output of cell 72
Unique specialties before: 71
Kept: ['InternalMedicine', 'Family/GeneralPractice', 'Emergency/Trauma', 'Cardiology',
'Surgery-General', 'Orthopedics', 'Orthopedics-Reconstructive', 'Radiologist', 'Nephrology',
 'Pulmonology']
Unique specialties after: 12
```

Figure 30-out: output (cell 72)

Step 12: Create new features (decisions 13 to 16)

Done: four features were built from domain knowledge:

Feature	How	Why	Result
total_visits	outpatient + emergency + inpatient visits	how much hospital care was needed in the past year	mean 0.56, max 49
age_mid	middle of the age band, [70-80) -> 75	turns the text band into a number	mean 65.44

Feature	How	Why	Result
num_active_meds	number of the 21 medicines not equal to No	how complex the treatment is	mean 1.19, max 6
num_med_changes	number of medicines with Up or Down	how much the treatment was adjusted	mean 0.26, max 4

Cell 74

```
# 1. total visits in the year before this stay
df["total_visits"] = df["number_outpatient"] + df["number_emergency"] + df["number_inpatient"]

# 2. age band -> middle number: "[70-80)" -> take "70" -> 70 + 5 = 75
df["age_mid"] = df["age"].str[1:].str.split("-").str[0].astype(int) + 5

# the medicine columns: everything between A1Cresult and change in the table
med_cols = df.loc[:, "metformin":"metformin-pioglitazone"].columns.tolist()
print(len(med_cols), "medicine columns:", med_cols)
```

Cell 75

```
# 3. how many medicines are being taken (value is not "No")
df["num_active_meds"] = (df[med_cols] != "No").sum(axis=1)

# 4. how many medicines had their dose changed (Up or Down)
df["num_med_changes"] = df[med_cols].isin(["Up", "Down"]).sum(axis=1)

df[["age", "age_mid", "total_visits", "num_active_meds", "num_med_changes"]].head()
```

Cell 76

```
df[["total_visits", "age_mid", "num_active_meds", "num_med_changes"]].describe()
```

Figure 31: Create total_visits, age_mid, num_active_meds, num_med_changes

Output of cell 74

```
21 medicine columns: ['metformin', 'repaglinide', 'nateglinide', 'chlorpropamide',
'glimepiride', 'acetohexamide', 'glipizide', 'glyburide', 'tolbutamide', 'pioglitazone',
'rosiglitazone', 'acarbose', 'miglitol', 'troglitazone', 'tolazamide', 'insulin',
'glyburide-metformin', 'glipizide-metformin', 'glimepiride-pioglitazone', 'metformin-
rosiglitazone', 'metformin-pioglitazone']
```

Output of cell 75

	age	age_mid	total_visits	num_active_meds	num_med_changes
8	[80-90)	85	0.0	2	0
9	[90-100)	95	0.0	2	0
4	[40-50)	45	0.0	2	0
10	[40-50)	45	0.0	1	0
5	[50-60)	55	0.0	1	0

Output of cell 76

	total_visits	age_mid	num_active_meds	num_med_changes
count	69970.000000	69970.000000	69970.000000	69970.000000
mean	0.559726	65.440046	1.190124	0.261884
std	1.427786	15.974203	0.942529	0.475849
min	0.000000	5.000000	0.000000	0.000000
25%	0.000000	55.000000	1.000000	0.000000
50%	0.000000	65.000000	1.000000	0.000000
75%	1.000000	75.000000	2.000000	0.000000
max	49.000000	95.000000	6.000000	4.000000

Figure 31-out: output (cells 74, 75, 76)

Step 13: Create the target (decision 17)

Done: readmitted_30 = 1 if readmitted is "<30", else 0. **Why:** the project question is 30-day readmission. **Result:** 6,277 positives (8.97%) against 63,693 (91.03%).

Cell 78

```
1 df["readmitted_30"] = (df["readmitted"] == "<30").astype(int)
2
3 print(df["readmitted_30"].value_counts())
4 print((df["readmitted_30"].value_counts(normalize=True) * 100).round(2))
```

Figure 32: Create the target readmitted_30

Output of cell 78

```
readmitted_30
0    63693
1     6277
Name: count, dtype: int64
readmitted_30
0    91.03
1     8.97
Name: proportion, dtype: float64
```

Figure 32-out: output (cell 78)

Step 14: Save the readable file (decision 18)

Done: cleaned_readable.csv was saved before encoding and scaling. **Why:** EDA charts and Power BI need real units (days, years) and text labels. **Result:** 69,970 x 51, 0 missing values.

Cell 80

```
1 # fresh row numbers (rows were sorted and dropped above)
2 df = df.reset_index(drop=True)
3
4 # final check: no missing values
5 print("Missing values:", df.isna().sum().sum())
6 print("Shape:", df.shape)
7
8 df.to_csv(f"{DATA_DIR}/cleaned_readable.csv", index=False)
9 print("Saved ../data/cleaned_readable.csv")
```

Figure 33: Save the readable file cleaned_readable.csv

Output of cell 80

```
Missing values: 0
Shape: (69970, 51)
Saved ../data/cleaned_readable.csv
```

Figure 33-out: output (cell 80)

Step 15: Encode categorical columns (decisions 19 to 22)

- **Label encoding** for two-value columns: gender (Male = 1), change (Ch = 1), diabetesMed (Yes = 1).

- **Medicine columns** (21): 0 = No, 1 = taken (Steady, Up or Down). The Up / Down direction is already kept in num_med_changes.
- **One-hot encoding** (drop_first=True) for categories with no order: race, payer_code, medical_specialty, A1Cresult, the 3 diagnosis groups and the 3 ID columns. The ID columns are turned into text first.
- **Dropped:** encounter_id, patient_nbr, age (text) and readmitted (text). IDs carry no medical meaning, and age and readmitted were replaced by age_mid and readmitted_30.

Result: 69,970 rows x 140 columns.

Cell 82

```
# start from a copy so the readable df stays as it is
model_df = df.copy()

# drop IDs and columns that were replaced by new ones
model_df = model_df.drop(columns=["encounter_id", "patient_nbr", "age", "readmitted"])

# label encoding for the 2-value columns
model_df["gender"] = (model_df["gender"] == "Male").astype(int)        # 1 = Male, 0 = Female
model_df["change"] = (model_df["change"] == "Ch").astype(int)          # 1 = medicine changed
model_df["diabetesMed"] = (model_df["diabetesMed"] == "Yes").astype(int)  # 1 = on diabetes medicine

# medicine columns: 1 = taking it, 0 = not
for col in med_cols:
    model_df[col] = (model_df[col] != "No").astype(int)

model_df[["gender", "change", "diabetesMed", "metformin", "insulin"]].head()
```

Cell 83

```
# ID codes are numbers but mean categories -> turn them into text first so get_dummies treats them as categories
for col in ["admission_type_id", "discharge_disposition_id", "admission_source_id"]:
    model_df[col] = model_df[col].astype(str)

one_hot_cols = ["race", "payer_code", "medical_specialty", "A1Cresult",
                "diag_1_group", "diag_2_group", "diag_3_group",
                "admission_type_id", "discharge_disposition_id", "admission_source_id"]

model_df = pd.get_dummies(model_df, columns=one_hot_cols, drop_first=True, dtype=int)

print("Shape after encoding:", model_df.shape)
```

Figure 34: Label and one-hot encode the categorical columns

Output of cell 82

	gender	change	diabetesMed	metformin	insulin
8	0	1	1	0	1
9	0	1	1	0	1
4	1	1	1	0	1
10	0	0	1	0	1
5	1	0	1	0	1

Output of cell 83

```
Shape after encoding: (69970, 140)
```

Figure 34-out: output (cells 82, 83)

Step 16: Scale numeric columns (decision 23)

Done: StandardScaler was applied to 12 numeric columns. The 0/1 columns and the target were not scaled. **Why:** the columns have very different ranges, and logistic regression is sensitive to

scale. **Result:** every scaled column has mean 0.0 and std 1.0 (a mean of -0.0 is 0 rounded to 2 decimals).

Cell 85

```
scale_cols = ["time_in_hospital", "num_lab_procedures", "num_procedures", "num_medications",
              "number_outpatient", "number_emergency", "number_inpatient", "number_diagnoses",
              "total_visits", "age_mid", "num_active_meds", "num_med_changes"]

scaler = StandardScaler()
model_df[scale_cols] = scaler.fit_transform(model_df[scale_cols])

# check: mean should be ~0 and std ~1
model_df[scale_cols].describe().loc[["mean", "std"]].round(2)
```

Figure 35: Standard-scale 12 numeric columns and check mean and std

Output of cell 85

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	\
mean	-0.0	0.0	0.0	0.0	
std	1.0	1.0	1.0	1.0	

	number_outpatient	number_emergency	number_inpatient	number_diagnoses	\
mean	0.0	0.0	0.0	-0.0	
std	1.0	1.0	1.0	1.0	

	total_visits	age_mid	num_active_meds	num_med_changes
mean	0.0	-0.0	-0.0	-0.0
std	1.0	1.0	1.0	1.0

Figure 35-out: output (cell 85)

Known limitation. The scaler is fit on all 69,970 rows before the train/test split in notebook 04, so mean and standard deviation leak a tiny amount of test-set information. This is accepted because the task asks for scaling in the cleaning step.

Step 17: Save cleaned.csv

Done: a final check before saving the model file. **Result:** 0 missing values, 69,970 x 140, target rate 8.97%.

Cell 87

```
print("Missing values:", model_df.isna().sum().sum())
print("Final shape:", model_df.shape)
print("Target rate (%):", round(model_df["readmitted_30"].mean() * 100, 2))

model_df.to_csv(f"{DATA_DIR}/cleaned.csv", index=False)
print("Saved ../data/cleaned.csv")
```

Figure 36: Final check and save cleaned.csv



Figure 36-out: output (cell 87)

3. Exploratory Data Analysis

Notebook Part 3. Input: the readable table df from Part 2 (69,970 patients, one encounter each, real units). Overall 30-day readmission rate: **8.97%** (6,277 of 69,970 patients).

3.1 Descriptive statistics

Cell 91

```
# overall readmission rate, used as a reference line in the charts
overall_rate = df["readmitted_30"].mean() * 100

print("Shape:", df.shape)
print("Overall 30-day readmission rate:", round(overall_rate, 2), "%")
```

Figure 37: Shape and overall 30-day readmission rate

Output of cell 91

```
Shape: (69970, 51)
Overall 30-day readmission rate: 8.97 %
```

Figure 37-out: output (cell 91)

Numeric columns. The mean stay is 4.24 days, the mean age (middle of band) is 65.44 years and the mean number of medications is 15.43. Prior visits are mostly 0: the median of number_outpatient, number_emergency and number_inpatient is 0.

Cell 93

```
numeric_cols = ["time_in_hospital", "num_lab_procedures", "num_procedures", "num_medications",
                "number_outpatient", "number_emergency", "number_inpatient", "number_diagnoses",
                "total_visits", "age_mid", "num_active_meds", "num_med_changes"]

# count, mean, std, min, quartiles, max of every numeric column
df[numeric_cols].describe().T.round(2)
```

Figure 38: Describe table of the numeric columns

Output of cell 93

	count	mean	std	min	25%	50%	75%	max
time_in_hospital	69970.0	4.24	2.85	1.0	2.0	3.0	6.0	12.0
num_lab_procedures	69970.0	42.87	19.87	1.0	31.0	44.0	57.0	96.0
num_procedures	69970.0	1.37	1.62	0.0	0.0	1.0	2.0	5.0
num_medications	69970.0	15.43	7.47	1.0	10.0	14.0	20.0	35.0
number_outpatient	69970.0	0.28	1.06	0.0	0.0	0.0	0.0	42.0
number_emergency	69970.0	0.10	0.51	0.0	0.0	0.0	0.0	42.0
number_inpatient	69970.0	0.18	0.60	0.0	0.0	0.0	0.0	12.0
number_diagnoses	69970.0	7.22	2.00	1.0	6.0	8.0	9.0	13.5
total_visits	69970.0	0.56	1.43	0.0	0.0	0.0	1.0	49.0
age_mid	69970.0	65.44	15.97	5.0	55.0	65.0	75.0	95.0
num_active_meds	69970.0	1.19	0.94	0.0	1.0	1.0	2.0	6.0
num_med_changes	69970.0	0.26	0.48	0.0	0.0	0.0	0.0	4.0

Figure 38-out: output (cell 93)

Categorical columns. The most common values are Caucasian (52,292), Female (37,229), age [70-80) (17,748), A1Cresult "Not tested" (57,125) and Circulatory as primary diagnosis (21,383). medical_specialty is Unknown for 33,637 patients.

Cell 94

```
# categorical columns: number of unique values, most common value and how often it appears
cat_cols = ["race", "gender", "age", "A1Cresult", "change", "diabetesMed", "medical_specialty", "diag_1_group"]

df[cat_cols].describe().T
```

Figure 39: Describe table of the categorical columns

Output of cell 94

	count	unique	top	freq
race	69970	6	Caucasian	52292
gender	69970	2	Female	37229
age	69970	10	[70-80)	17748
A1Cresult	69970	4	Not tested	57125
change	69970	2	No	38481
diabetesMed	69970	2	Yes	53291
medical_specialty	69970	12	Unknown	33637
diag_1_group	69970	9	Circulatory	21383

Figure 39-out: output (cell 94)

Readmitted vs not readmitted. Readmitted patients had longer stays (4.76 vs 4.19 days), more medications (16.37 vs 15.33), more lab procedures (44.90 vs 42.67) and more prior inpatient visits (0.37 vs 0.16). Each difference that matters is tested in 3.3.

Cell 95

```
# mean of each numeric column for readmitted (1) vs not readmitted (0)
df.groupby("readmitted_30")[numeric_cols].mean().T.round(2)
```

Figure 40: Mean of each numeric column for readmitted vs not

Output of cell 95

readmitted_30	0	1
time_in_hospital	4.19	4.76
num_lab_procedures	42.67	44.90
num_procedures	1.37	1.37
num_medications	15.33	16.37
number_outpatient	0.28	0.31
number_emergency	0.10	0.15
number_inpatient	0.16	0.37
number_diagnoses	7.19	7.51
total_visits	0.53	0.83
age_mid	65.20	67.83
num_active_meds	1.19	1.22
num_med_changes	0.26	0.30

Figure 40-out: output (cell 95)

3.2 Visualizations

All charts were made with Matplotlib and Seaborn. Each chart is the PNG saved in the notebook output. The readmission rate charts by group are shown with the insights in 3.3.

Distributions

Histograms. time_in_hospital and total_visits are right-skewed: most stays are short and most patients had no prior visits. age_mid is left-skewed: most patients are 50 or older. The bar at 12 days comes from the IQR cap. number_diagnoses has a tall spike at 9.

Cell 98

```
# 6 histograms in a 2 x 3 grid
hist_cols = ["time_in_hospital", "num_lab_procedures", "num_medications", "number_diagnoses", "age_mid", "total_visits"]
x_labels = ["Days in hospital", "Number of lab procedures", "Number of medications",
            "Number of diagnoses", "Age (middle of band, years)", "Prior visits (past year)"]

fig, axes = plt.subplots(2, 3, figsize=(14, 7))

# axes.flatten() turns the 2x3 grid into a simple list of 6 charts
for ax, col, label in zip(axes.flatten(), hist_cols, x_labels):
    ax.hist(df[col], bins=20, color="steelblue", edgecolor="white")
    ax.set_title("Distribution of " + col)
    ax.set_xlabel(label)
    ax.set_ylabel("Number of patients")

plt.tight_layout()
plt.show()
```

Figure 41: Histograms of six numeric columns

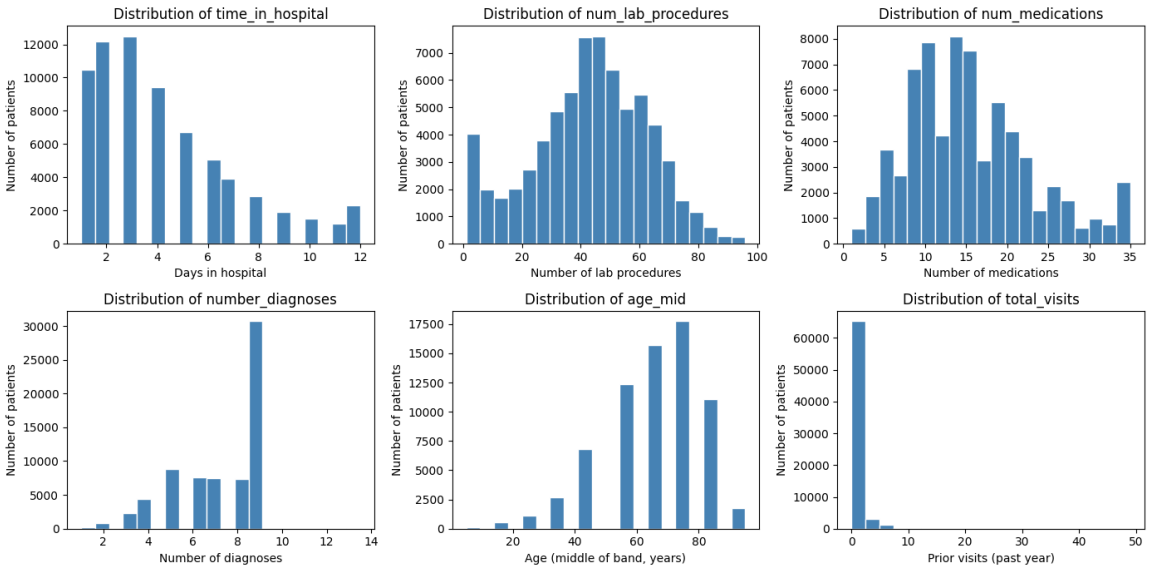


Figure 41-chart: output chart, Histograms of six numeric columns (cell 98)

Density plot. The number of medications has the same shape for both groups. The readmitted group has a slightly higher mean (16.37 vs 15.33).

Cell 100

```
# density plot: number of medications for readmitted vs not readmitted
plt.figure(figsize=(8, 4))
sns.kdeplot(data=df, x="num_medications", hue="readmitted_30", common_norm=False, fill=True)
plt.title("Number of Medications by Readmission Status (density)")
plt.xlabel("Number of medications")
plt.ylabel("Density")
plt.show()
```

Figure 42: Density plot of medications by readmission status

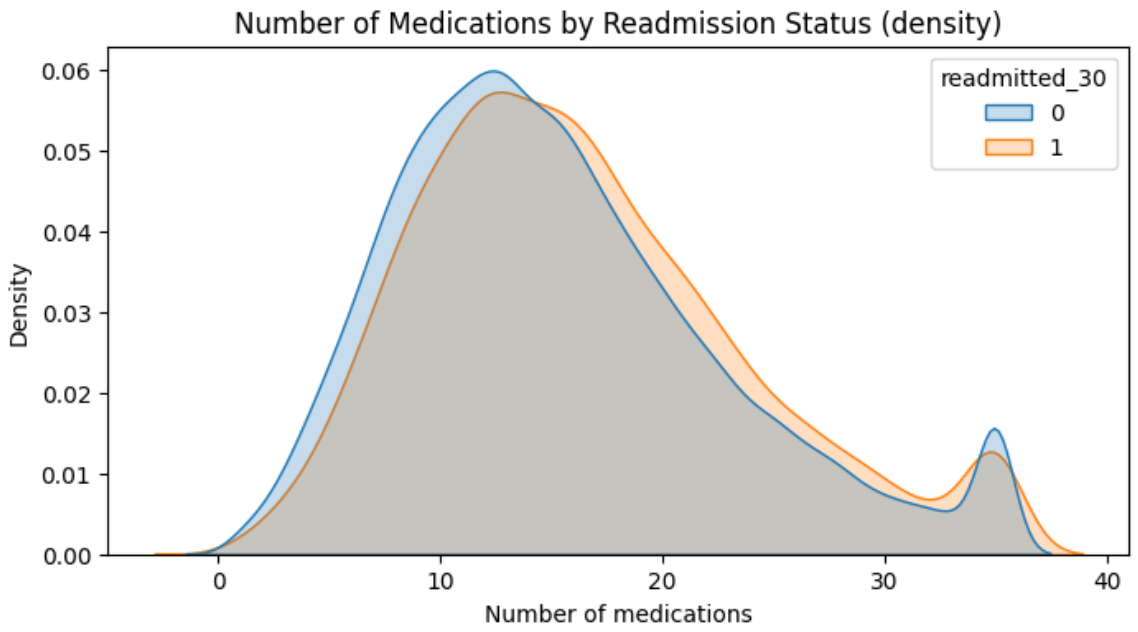


Figure 42-chart: output chart, Number of Medications by Readmission Status (density) (cell 100)

Relationships between variables

Scatter plot (random sample of 3,000 patients). Longer stays come with more medications. Readmitted and non-readmitted patients overlap.

Cell 118

```
# a random sample of 3,000 patients keeps the scatter plot readable
sample = df.sample(3000, random_state=42)

plt.figure(figsize=(8, 5))
sns.scatterplot(data=sample, x="time_in_hospital", y="num_medications", hue="readmitted_30", alpha=0.5)
plt.title("Length of Stay vs Number of Medications (sample of 3,000)")
plt.xlabel("Days in hospital")
plt.ylabel("Number of medications")
plt.show()
```

Figure 43: Scatter plot of stay vs medications

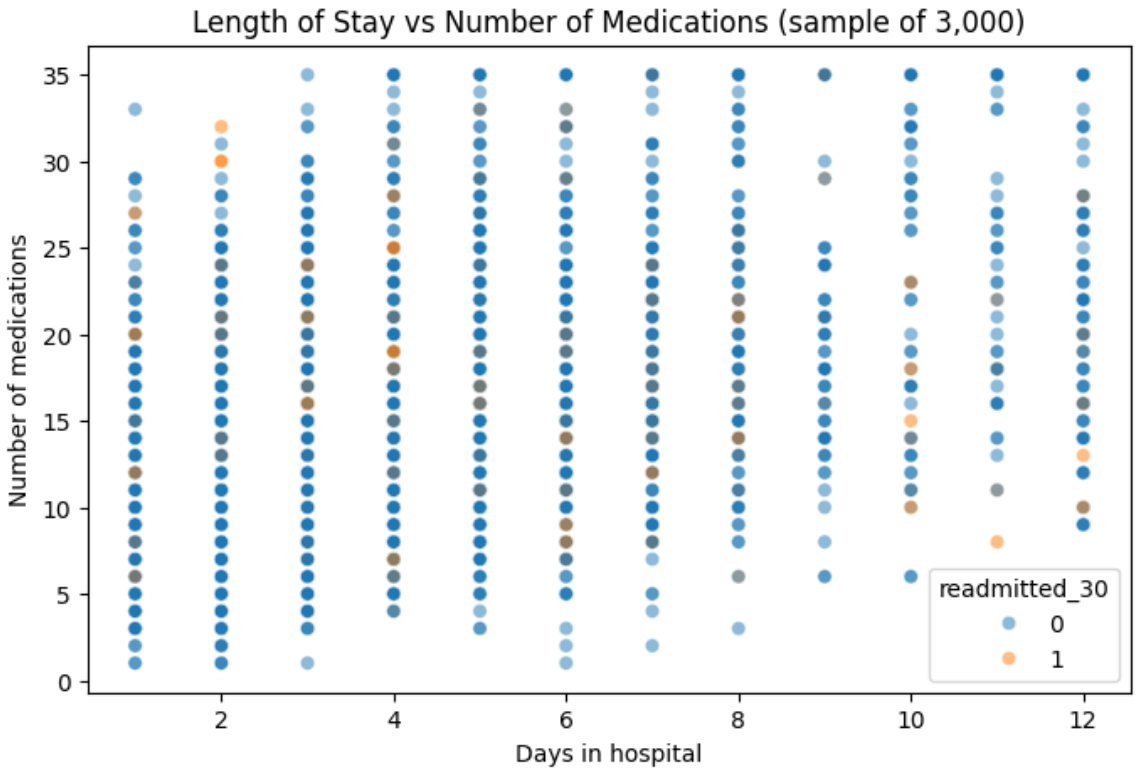


Figure 43-chart: output chart, Length of Stay vs Number of Medications (sample of 3,000) (cell 118)

Pair plot of four numeric columns on the same sample. The two groups overlap almost completely in every pair, so no single pair of variables separates them.

Cell 119

```
# pair plot: every pair of 4 columns, colored by readmission
pair = sns.pairplot(sample[["time_in_hospital", "num_medications", "num_lab_procedures", "total_visits", "readmitted_30"]],
                    hue="readmitted_30", plot_kws={"alpha": 0.4}, height=2.2)
# turn the x-axis numbers so they do not overlap
for ax in pair.axes.flatten():
    ax.tick_params(axis="x", rotation=45)
pair.figure.suptitle("Pair Plot of Key Numeric Columns (sample of 3,000)", y=1.02)
plt.show()
```

Figure 44: Pair plot of four numeric columns

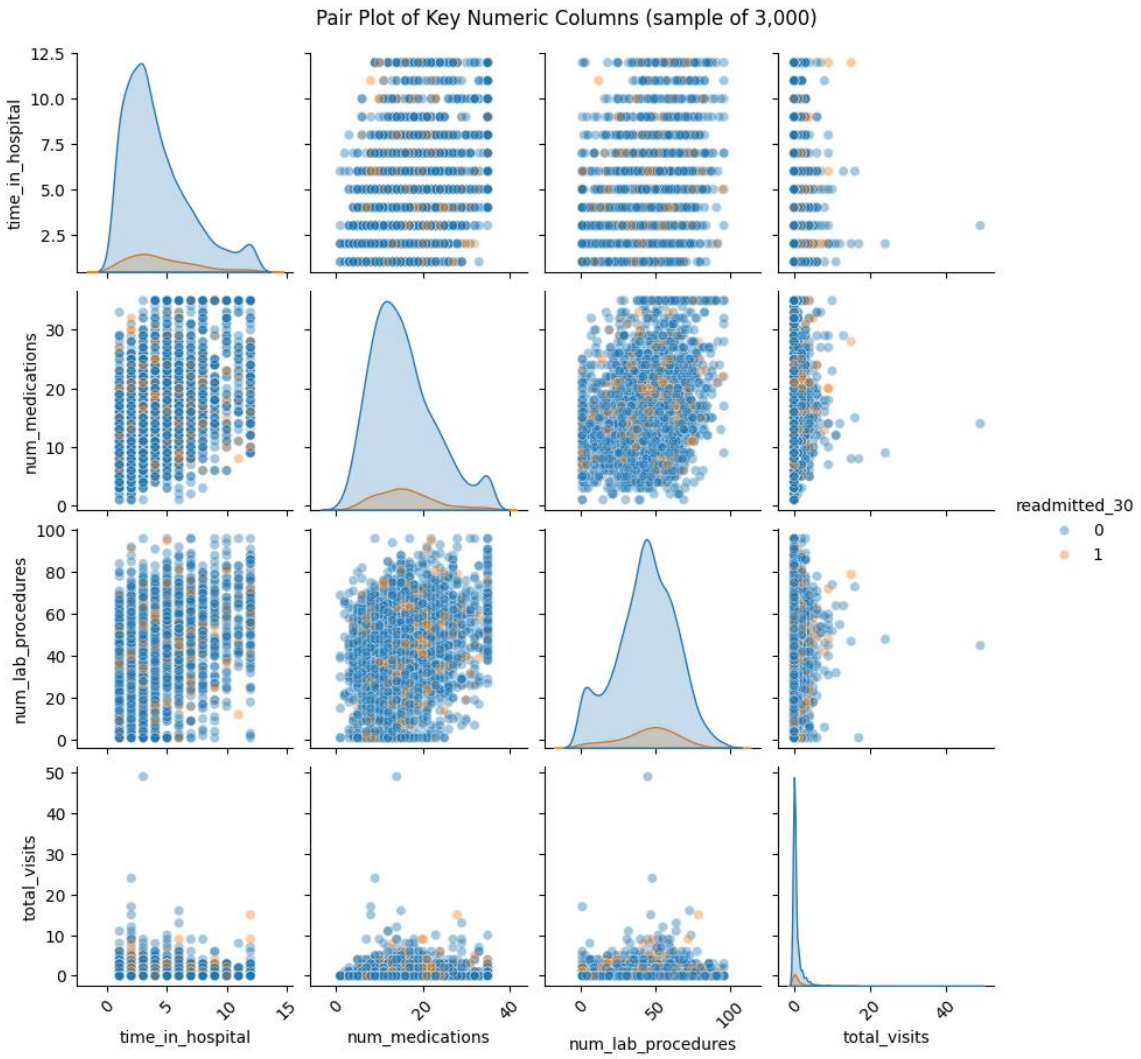


Figure 44-chart: output chart, Pair Plot of Key Numeric Columns (sample of 3,000) (cell 119)

Correlations

Heatmap and correlation with the target. All correlations with readmitted_30 are weak. The highest are number_inpatient (0.100), total_visits (0.059) and time_in_hospital (0.057).

Cell 122

```
corr = df[numeric_cols + ["readmitted_30"]].corr()

plt.figure(figsize=(11, 8))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Heatmap of Numeric Features and Target")
plt.xlabel("Feature")
plt.ylabel("Feature")
plt.show()
```

Figure 45: Correlation heatmap

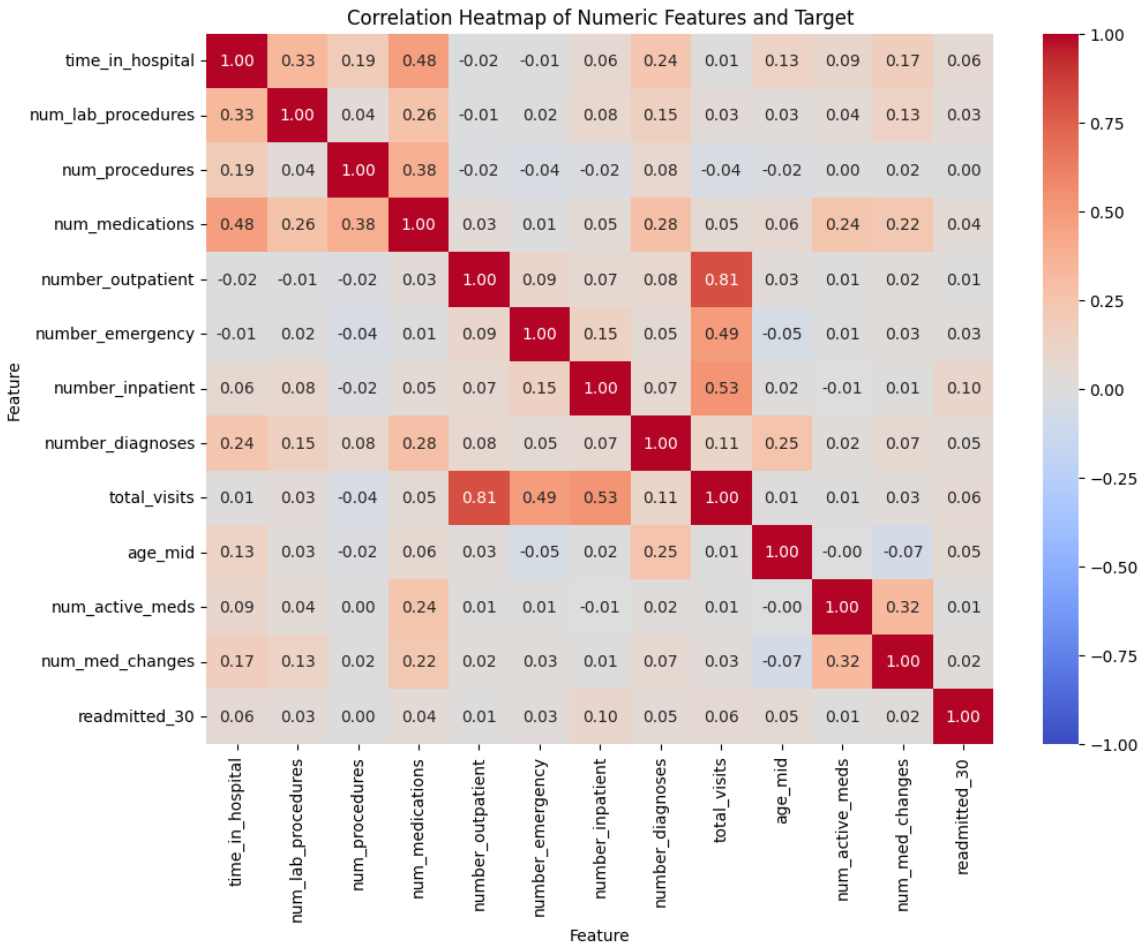


Figure 45-chart: output chart, Correlation Heatmap of Numeric Features and Target (cell 122)

Cell 123

```
# correlation of every feature with the target, strongest first
corr["readmitted_30"].drop("readmitted_30").sort_values(ascending=False).round(3)
```

Figure 46: Correlation of each feature with the target

Output of cell 123

```
number_inpatient      0.100
total_visits          0.059
time_in_hospital      0.057
age_mid               0.047
number_diagnoses      0.045
num_medications       0.039
num_lab_procedures    0.032
number_emergency      0.028
num_med_changes       0.023
num_active_meds       0.012
number_outpatient     0.009
num_procedures        0.001
Name: readmitted_30, dtype: float64
```

Figure 46-out: output (cell 123)

3.3 Key insights

These insights are sections 2.1 to 2.8 of the existing report. Each one is followed by the code and output that support it.

Insight 1: Prior inpatient visits are the strongest signal

Inpatient visits in the year before	Patients	Readmitted <30 days
0	61,779	8.12%
1	5,794	12.89%
2	1,501	18.52%
3+	896	26.45%

- The t-test is significant ($p \approx 4.7e\text{-}62$). Readmitted patients averaged 0.369 prior inpatient visits vs 0.157 for the rest.
- number_inpatient has the highest correlation with the target (0.100).
- **So what:** patients with 2 or more admissions in the past year should be first in line for discharge follow-up.

Cell 107

```
# group 3 or more prior inpatient visits together (the groups get small)
df["inpatient_band"] = df["number_inpatient"].clip(upper=3).astype(int).astype(str)
df.loc[df["inpatient_band"] == "3", "inpatient_band"] = "3+"

df["inpatient_band"].value_counts()
```

Figure 47: Group prior inpatient visits into bands

Output of cell 107

```
inpatient_band
0     61779
1      5794
2      1501
3+      896
Name: count, dtype: int64
```

Figure 47-out: output (cell 107)

Cell 108

```python
# readmission rate (%) and number of patients for each group
rate = df.groupby("inpatient_band")["readmitted_30"].agg(["count", "mean"])
rate["rate_pct"] = (rate["mean"] * 100).round(2)
print(rate[["count", "rate_pct"]])

plt.figure(figsize=(9, 4))
plt.bar(rate.index.astype(str), rate["rate_pct"])
plt.axhline(overall_rate, color="red", linestyle="--", label=f"Overall {overall_rate:.2f}%")
plt.title("30-Day Readmission Rate by Prior Inpatient Visits")
plt.xlabel("Inpatient visits in the year before")
plt.ylabel("Readmitted within 30 days (%)")
plt.xticks(rotation=0)
plt.legend()
plt.show()
```

Figure 48: Readmission rate by prior inpatient visits

Output of cell 108

inpatient_band	count	rate_pct
0	61779	8.12
1	5794	12.89
2	1501	18.52
3+	896	26.45

Figure 48-out: output (cell 108)

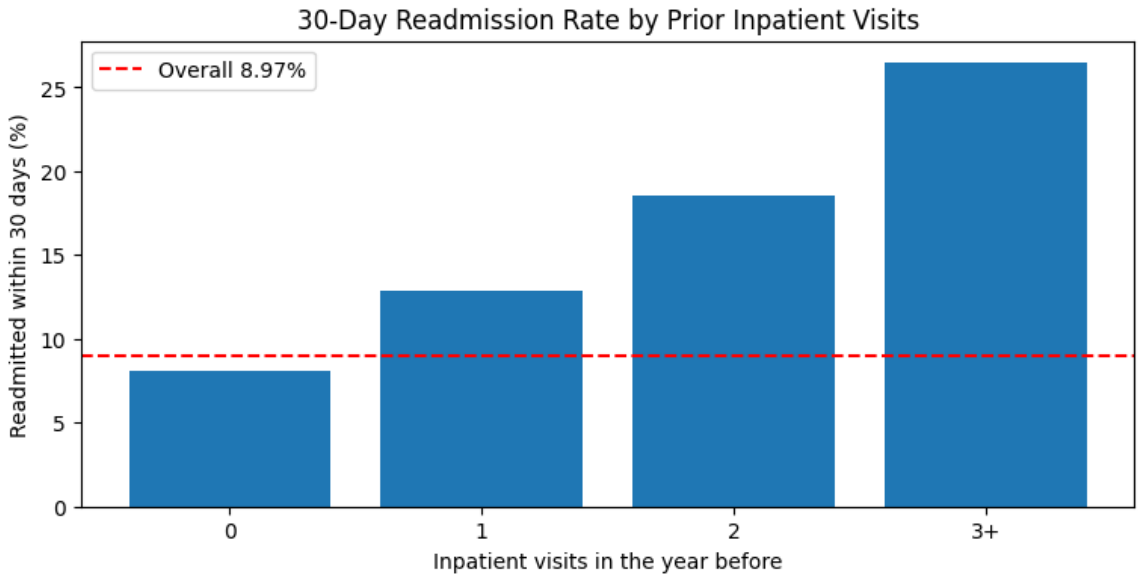


Figure 48-chart: output chart, 30-Day Readmission Rate by Prior Inpatient Visits (cell 108)

Cell 139

```
readmitted = df.loc[df["readmitted_30"] == 1, "number_inpatient"]
not_readmitted = df.loc[df["readmitted_30"] == 0, "number_inpatient"]

print("Mean prior inpatient visits, readmitted:", round(readmitted.mean(), 3))
print("Mean prior inpatient visits, not readmitted:", round(not_readmitted.mean(), 3))

t_stat, p = ttest_ind(readmitted, not_readmitted, equal_var=False)
print("\nt-statistic:", round(t_stat, 2), "| p-value:", p)
print("Reject H0" if p < 0.05 else "Do not reject H0")
```

Figure 49: Welch t-test on prior inpatient visits

Output of cell 139

```
Mean prior inpatient visits, readmitted: 0.369
Mean prior inpatient visits, not readmitted: 0.157

t-statistic: 16.8 | p-value: 4.655913786179672e-62
Reject H0
```

Figure 49-out: output (cell 139)

Insight 2: Longer stays carry higher risk

- The rate rises from 6.54% for 1-day stays to 12.32% for 8-day stays (12.28%-12.30% at 9-10 days).
- The t-test is significant ($p \approx 2.6e\text{-}47$). Mean stay was 4.76 days for readmitted vs 4.19 days for the rest.

Cell 110

```
# readmission rate (%) and number of patients for each group
rate = df.groupby("time_in_hospital")["readmitted_30"].agg(["count", "mean"])
rate["rate_pct"] = (rate["mean"] * 100).round(2)
print(rate[["count", "rate_pct"]])

plt.figure(figsize=(9, 4))
plt.bar(rate.index.astype(str), rate["rate_pct"])
plt.axhline(overall_rate, color="red", linestyle="--", label=f"Overall {overall_rate:.2f}%")
plt.title("30-Day Readmission Rate by Length of Stay")
plt.xlabel("Days in hospital (12 = 12 or more)")
plt.ylabel("Readmitted within 30 days (%)")
plt.xticks(rotation=0)
plt.legend()
plt.show()
```

Figure 50: Readmission rate by length of stay

Output of cell 110

time_in_hospital	count	rate_pct
1.0	10468	6.54
2.0	12178	7.66
3.0	12487	8.18
4.0	9423	8.86
5.0	6697	9.77
6.0	5050	10.89
7.0	3902	11.43
8.0	2842	12.32
9.0	1911	12.30
10.0	1506	12.28
11.0	1188	9.60
12.0	2318	11.56

Figure 50-out: output (cell 110)

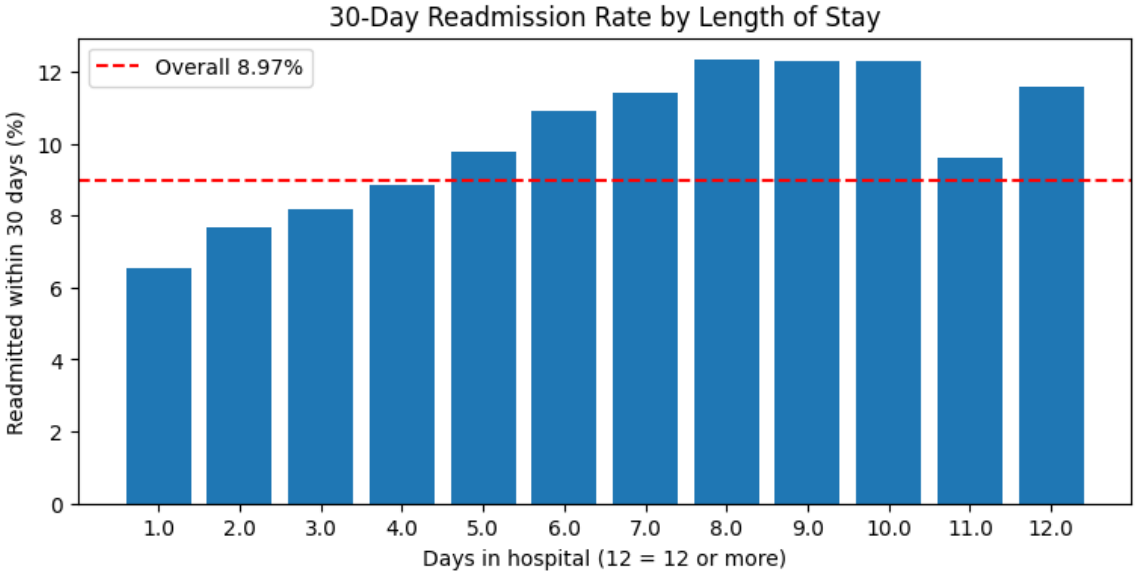


Figure 50-chart: output chart, 30-Day Readmission Rate by Length of Stay (cell 110)

Cell 136

```
readmitted = df.loc[df["readmitted_30"] == 1, "time_in_hospital"]
not_readmitted = df.loc[df["readmitted_30"] == 0, "time_in_hospital"]

print("Mean days, readmitted:", round(readmitted.mean(), 3))
print("Mean days, not readmitted:", round(not_readmitted.mean(), 3))

t_stat, p = ttest_ind(readmitted, not_readmitted, equal_var=False)
print("\nt-statistic:", round(t_stat, 2), "| p-value:", p)
print("Reject H0" if p < 0.05 else "Do not reject H0")
```

Figure 51: Welch t-test on length of stay

Output of cell 136

```
Mean days, readmitted: 4.761
Mean days, not readmitted: 4.193

t-statistic: 14.55 | p-value: 2.568817065064052e-47
Reject H0
```

Figure 51-out: output (cell 136)

Insight 3: Older patients are at higher risk

- Ages 70-80: 10.24%, ages 80-90: 10.76% (the highest bands).
- Ages 30-60 sit around 7% (6.98%-7.41%).

Cell 103

```python
# readmission rate (%) and number of patients for each group
rate = df.groupby("age")["readmitted_30"].agg(["count", "mean"])
rate["rate_pct"] = (rate["mean"] * 100).round(2)
print(rate[["count", "rate_pct"]])

plt.figure(figsize=(9, 4))
plt.bar(rate.index.astype(str), rate["rate_pct"])
plt.axhline(overall_rate, color="red", linestyle="--", label=f"Overall {overall_rate:.2f}%")
plt.title("30-Day Readmission Rate by Age Band")
plt.xlabel("Age band")
plt.ylabel("Readmitted within 30 days (%)")
plt.xticks(rotation=0)
plt.legend()
plt.show()
```

Figure 52: Readmission rate by age band

Output of cell 103

```
          count  rate_pct
age
[0-10)      153      1.96
[10-20)     534      4.87
[20-30)    1121      7.40
[30-40)    2692      6.98
[40-50)    6828      7.41
[50-60)   12349      7.12
[60-70)   15683      9.00
[70-80)   17748     10.24
[80-90)   11102     10.76
[90-100)   1760      9.55
```

Figure 52-out: output (cell 103)

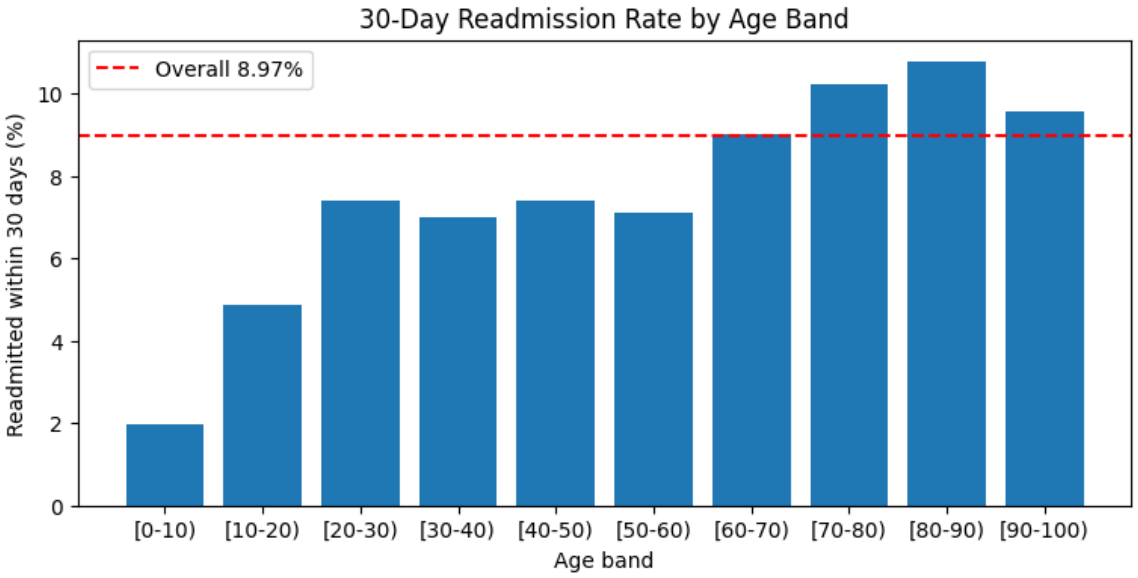


Figure 52-chart: output chart, 30-Day Readmission Rate by Age Band (cell 103)

Insight 4: Primary diagnosis matters

- Highest: Injury 10.78%, Circulatory 9.67%, Diabetes 9.12%.
- Lowest: Respiratory 7.28%, Digestive 8.02%.
- Circulatory is also the largest group (21,383 patients, 31% of all).
- The chi-square test is significant ($\chi^2 = 73.7$, dof = 8, $p \approx 9.0e\text{-}13$).

Cell 105

```
# readmission rate (%) and number of patients for each group
rate = df.groupby("diag_1_group")["readmitted_30"].agg(["count", "mean"])
rate["rate_pct"] = (rate["mean"] * 100).round(2)
rate = rate.sort_values("rate_pct", ascending=False)
print(rate[["count", "rate_pct"]])

plt.figure(figsize=(9, 4))
plt.bar(rate.index.astype(str), rate["rate_pct"])
plt.axhline(overall_rate, color="red", linestyle="--", label=f"Overall {overall_rate:.2f}%")
plt.title("30-Day Readmission Rate by Primary Diagnosis")
plt.xlabel("Primary diagnosis group")
plt.ylabel("Readmitted within 30 days (%)")
plt.xticks(rotation=30)
plt.legend()
plt.show()
```

Figure 53: Readmission rate by primary diagnosis group

Output of cell 105

diag_1_group	count	rate_pct
Injury	4692	10.78
Circulatory	21383	9.67
Diabetes	5748	9.12
Neoplasms	2538	9.06
Genitourinary	3440	8.98
Other	12132	8.98
Musculoskeletal	4064	8.39
Digestive	6487	8.02
Respiratory	9486	7.28

Figure 53-out: output (cell 105)

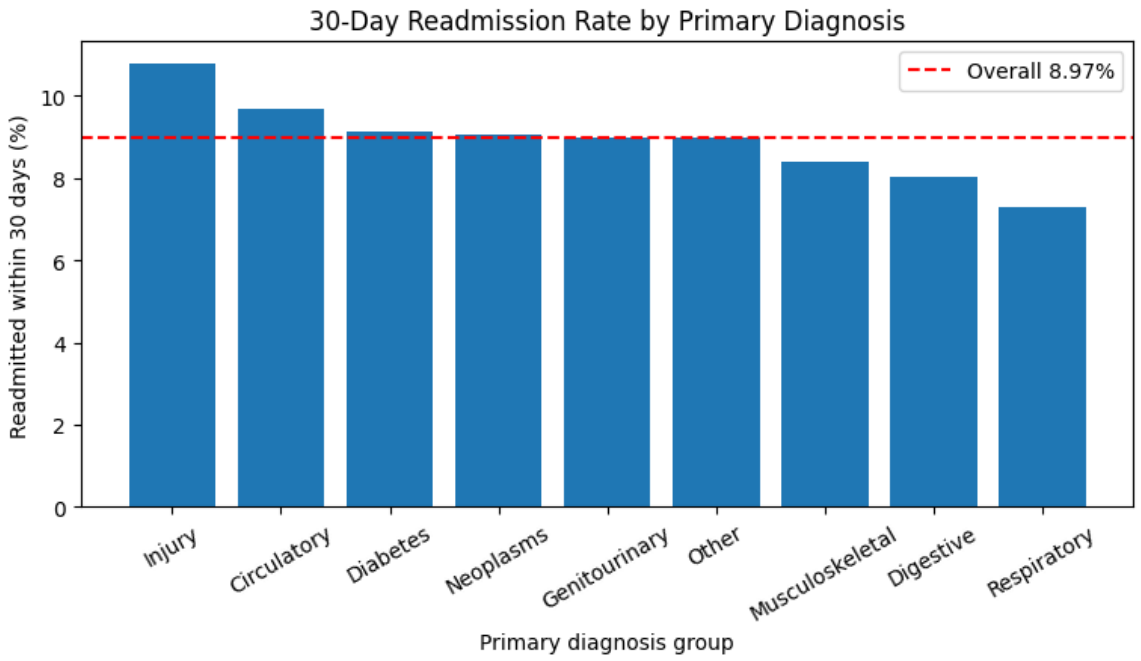


Figure 53-chart: output chart, 30-Day Readmission Rate by Primary Diagnosis (cell 105)

Cell 130

```
1 table = pd.crosstab(df["diag_1_group"], df["readmitted_30"])
2
3 chi2, p, dof, expected = chi2_contingency(table)
4 print("Chi-square:", round(chi2, 2), "| dof:", dof, "| p-value:", p)
5 print("Reject H0" if p < 0.05 else "Do not reject H0")
```

Figure 54: Chi-square test, diagnosis group vs readmission

Output of cell 130

```
Chi-square: 73.7 | dof: 8 | p-value: 8.984853507485246e-13
Reject H0
```

Figure 54-out: output (cell 130)

Insight 5: Medication factors are small but real

- Medication changed during the stay: 9.44% vs 8.59% not changed ($\chi^2 = 15.0$, $p = 0.0001$).

- On diabetes medication: 9.42% vs 7.55% not on it.

Cell 114

```
# readmission rate (%) and number of patients for each group
rate = df.groupby("change")["readmitted_30"].agg(["count", "mean"])
rate["rate_pct"] = (rate["mean"] * 100).round(2)
print(rate[["count", "rate_pct"]])

plt.figure(figsize=(9, 4))
plt.bar(rate.index.astype(str), rate["rate_pct"])
plt.axhline(overall_rate, color="red", linestyle="--", label=f"Overall {overall_rate:.2f}%")
plt.title("30-Day Readmission Rate by Medication Change")
plt.xlabel("Medication changed during stay (Ch = yes)")
plt.ylabel("Readmitted within 30 days (%)")
plt.xticks(rotation=0)
plt.legend()
plt.show()
```

Figure 55: Readmission rate by medication change

Output of cell 114

```
        count  rate_pct
change
Ch      31489      9.44
No      38481      8.59
```

Figure 55-out: output (cell 114)

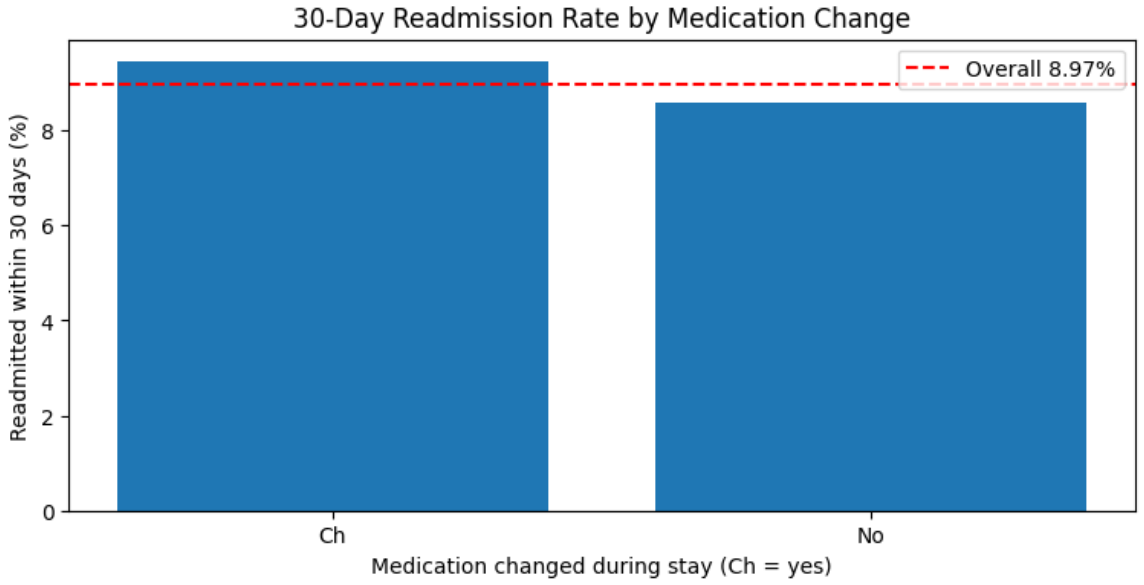


Figure 55-chart: output chart, 30-Day Readmission Rate by Medication Change (cell 114)

Cell 115

```
# readmission rate (%) and number of patients for each group
rate = df.groupby("diabetesMed")["readmitted_30"].agg(["count", "mean"])
rate["rate_pct"] = (rate["mean"] * 100).round(2)
print(rate[["count", "rate_pct"]])

plt.figure(figsize=(9, 4))
plt.bar(rate.index.astype(str), rate["rate_pct"])
plt.axhline(overall_rate, color="red", linestyle="--", label=f"Overall {overall_rate:.2f}%")
plt.title("30-Day Readmission Rate by Diabetes Medication")
plt.xlabel("Diabetes medication prescribed")
plt.ylabel("Readmitted within 30 days (%)")
plt.xticks(rotation=0)
plt.legend()
plt.show()
```

Figure 56: Readmission rate by diabetes medication

Output of cell 115

diabetesMed	count	rate_pct
No	16679	7.55
Yes	53291	9.42

Figure 56-out: output (cell 115)

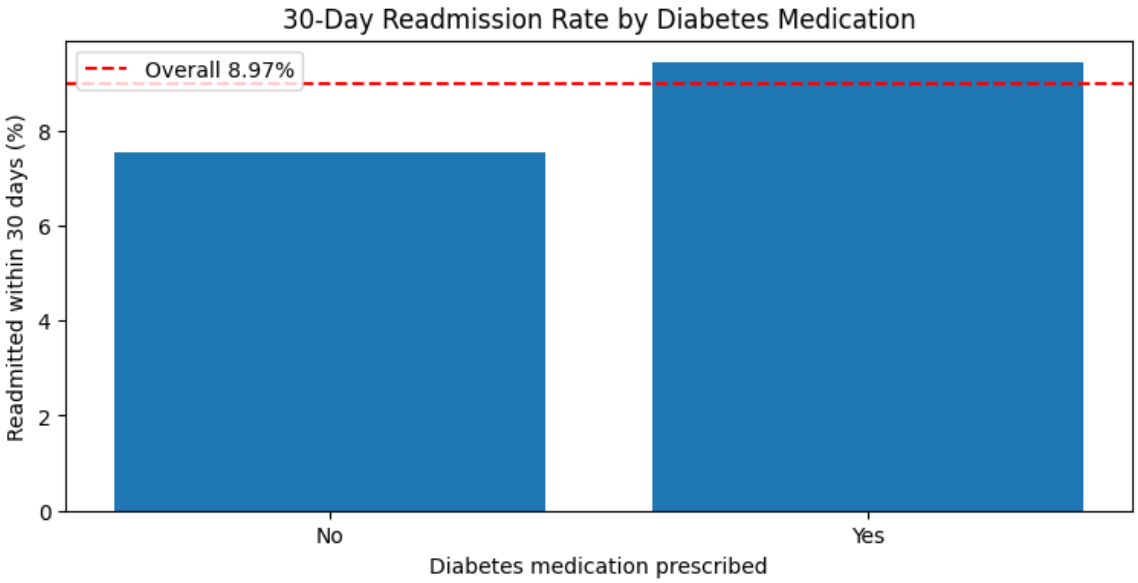


Figure 56-chart: output chart, 30-Day Readmission Rate by Diabetes Medication (cell 115)

Cell 127

```
table = pd.crosstab(df["change"], df["readmitted_30"])
print(table)

chi2, p, dof, expected = chi2_contingency(table)
print("\nChi-square:", round(chi2, 2), "| dof:", dof, "| p-value:", p)
print("Reject H0" if p < 0.05 else "Do not reject H0")
```

Figure 57: Chi-square test, medication change vs readmission

```
Output of cell 127
```

```
readmitted_30      0     1
change
Ch             28518  2971
No             35175  3306

Chi-square: 15.0 | dof: 1 | p-value: 0.00010776426381830976
Reject H0
```

Figure 57-out: output (cell 127)

Insight 6: HbA1c testing showed no significant effect

- Rates: >7 = 8.59%, >8 = 8.16%, Norm = 8.63%, Not tested = 9.10%.
- Chi-square: $\chi^2 = 7.27$, dof = 3, p = 0.064 > 0.05, so not significant.
- 57,125 patients (82%) were never tested, so this result says little about the test itself.

```
Cell 112
```

```python
# readmission rate (%) and number of patients for each group
rate = df.groupby("A1Cresult")["readmitted_30"].agg(["count", "mean"])
rate["rate_pct"] = (rate["mean"] * 100).round(2)
print(rate[["count", "rate_pct"]])

plt.figure(figsize=(9, 4))
plt.bar(rate.index.astype(str), rate["rate_pct"])
plt.axhline(overall_rate, color="red", linestyle="--", label=f"Overall {overall_rate:.2f}%")
plt.title("30-Day Readmission Rate by HbA1c Test Result")
plt.xlabel("HbA1c result")
plt.ylabel("Readmitted within 30 days (%)")
plt.xticks(rotation=0)
plt.legend()
plt.show()
```

Figure 58: Readmission rate by HbA1c result

```
Output of cell 112
```

```
            count  rate_pct
A1Cresult
>7           2865      8.59
>8           6239      8.16
Norm         3741      8.63
Not tested  57125      9.10
```

Figure 58-out: output (cell 112)

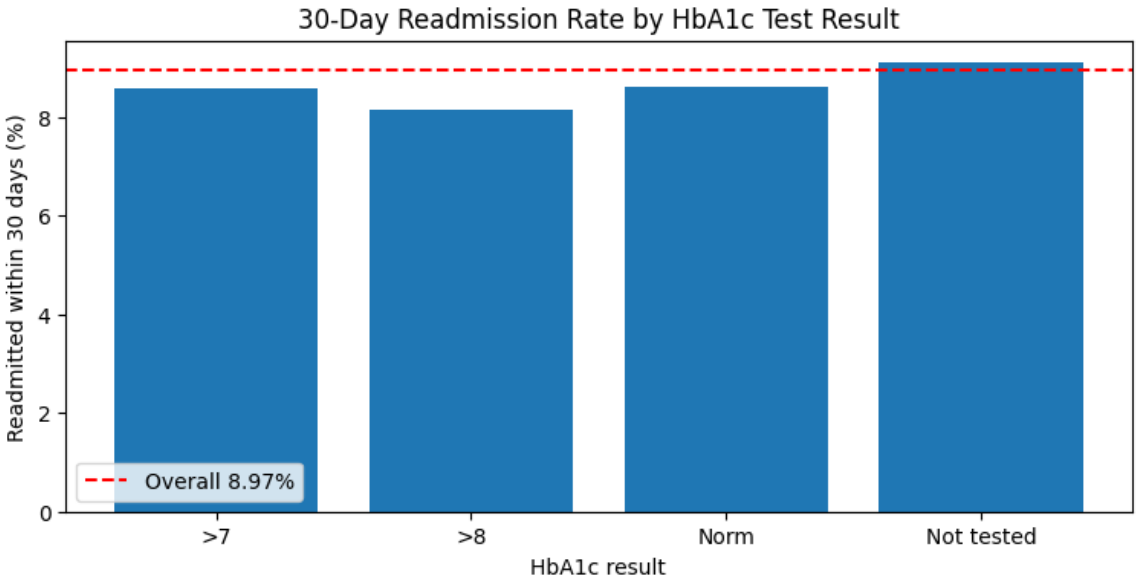


Figure 58-chart: output chart, 30-Day Readmission Rate by HbA1c Test Result (cell 112)

Cell 133

```
table = pd.crosstab(df["A1Cresult"], df["readmitted_30"])
print(table)

chi2, p, dof, expected = chi2_contingency(table)
print("\nChi-square:", round(chi2, 2), "| dof:", dof, "| p-value:", p)
print("Reject H0" if p < 0.05 else "Do not reject H0")
```

Figure 59: Chi-square test, HbA1c result vs readmission

Output of cell 133

```
readmitted_30      0     1
A1Cresult
>7              2619   246
>8              5730   509
Norm            3418   323
Not tested     51926  5199

Chi-square: 7.27 | dof: 3 | p-value: 0.06381865444870938
Do not reject H0
```

Figure 59-out: output (cell 133)

Insight 7: No single variable separates the groups

- All correlations with the target are weak (max 0.100, Figure 46-out).
- Scatter and pair plots show readmitted and non-readmitted patients overlapping almost completely (Figure 43-chart and Figure 44-chart).
- Expectation for modeling: a model will need many features together, and precision on the positive class will be low.

Insight 8: Hypothesis test summary (alpha = 0.05)

Chi-square tests compare two categorical variables. Welch t-tests compare the mean of a numeric variable between readmitted and not readmitted patients and do not assume equal variances.

#	Test	Variables	Statistic	p-value	Result
1	Chi-square	change vs readmitted_30	$\chi^2 = 15.0$	0.0001	Significant
2	Chi-square	diag_1_group vs readmitted_30	$\chi^2 = 73.7$	9.0e-13	Significant
3	Chi-square	A1Cresult vs readmitted_30	$\chi^2 = 7.27$	0.064	Not significant
4	Welch t-test	time_in_hospital	t = 14.55	2.6e-47	Significant
5	Welch t-test	number_inpatient	t = 16.80	4.7e-62	Significant

Caution: with about 70,000 patients even small differences become significant. Effect sizes (the rate gaps above) matter more than p-values.

What this means for decisions and further analysis:

- Use prior inpatient visits first when choosing patients for discharge follow-up.
- Length of stay, age and primary diagnosis add smaller differences in risk.
- HbA1c results cannot guide this decision in this data, because most patients were never tested.
- A prediction model must combine many features, and its precision on the readmitted class will be low.

4. Conclusion

4.1 Process documentation

Dataset overview. UCI Diabetes 130-US Hospitals for Years 1999-2008. diabetic_data.csv has one row per hospital encounter (101,766 rows x 50 columns). IDS_mapping.csv holds the lookup tables for the admission type, discharge disposition and admission source codes. The target is readmission within 30 days (readmitted = "<30").

Data cleaning and transformation. 23 decisions, each with a reason and an effect (section 2). The "?" placeholders became NaN. Four columns were dropped (two over 90% missing, two constant). The rest of the missing values were filled with "Not tested" or "Unknown". One encounter per patient was kept, and expired, hospice and invalid gender rows were removed: 101,766 → 71,518 → 69,973 → 69,970 rows. Outliers were capped with the IQR rule. ICD-9 codes were grouped, rare specialties combined and four features created. The model file was encoded (140 columns) and 12 numeric columns were scaled.

EDA results and key findings. The 30-day readmission rate is 8.97%. Prior inpatient visits are the strongest signal (8.12% for 0 visits up to 26.45% for 3 or more). Longer stays, older age and some diagnoses (Injury, Circulatory) carry higher risk. Medication change has a small but significant effect. HbA1c testing has no significant effect. All correlations with the target are weak (max 0.100).

Where the documentation lives. The notebook explains every step in markdown. The notebook also refers to docs/cleaning_log.md (every cleaning decision) and docs/insights.md (EDA insights). This report collects all of it.

4.2 Code structure

The code is one Jupyter notebook with 189 cells (95 code, 94 markdown). It runs top to bottom. Everything is imported and loaded once in Setup, and each part reuses the variables of the part before it.

Part	What it does	Main variables
Setup	imports, paths, load all files	df_raw, lookup tables
1. Exploration	look at the raw data, no changes	df_raw
2. Cleaning	clean, build features, encode, scale	df (readable), model_df (encoded + scaled)
3. EDA	charts, group rates, hypothesis tests	df
4. Modeling	logistic regression and decision tree	results, top_features

- **Markdown before each step.** Each part opens with a markdown cell that lists its input, output and steps. Each step has a heading ("Step N: ...") and a short explanation before the code.
- **Results explained after the code.** "What we see" cells explain the output, and each hypothesis test has an "Interpretation" cell.
- **Comments in the code** explain the reason for each line that is not obvious, for example "# keep only rows whose discharge code is NOT in the list (~ means NOT)" (Figure 26) and "# make the columns float first, because an upper limit like 13.5 is not a whole number" (Figure 28).
- **Repeatable results.** The sample for the scatter and pair plots and the train/test split use random_state=42. The data folder is set once (DATA_DIR = "../data").

5. Version Control and Deliverables

5.1 Git and GitHub

Git is used to track the project. The repository is published on GitHub at https://github.com/ManilBista/healthcare-readmission-analysis. It holds the notebook, the cleaned datasets, the documentation and the README, so each change can be traced and the analysis can be repeated.

5.2 Repository organization

The tree lists only the folders and files named in the existing report or the notebook.

```
<repository root>/
├── data/
│   ├── diabetic_data.csv          raw data (input)
│   ├── IDS_mapping.csv            ID lookup tables (input)
│   ├── cleaned_readable.csv       69,970 x 51, for EDA and Power BI
│   ├── cleaned.csv                69,970 x 140, encoded and scaled
│   ├── model_metrics.csv          model results (Part 4)
│   ├── top_features.csv           top 10 features per model (Part 4)
│   └── powerbi/
│       ├── patients.csv
│       ├── readmission_by_group.csv
│       ├── model_metrics.csv
│       └── top_features.csv
├── docs/
│   ├── cleaning_log.md
│   ├── insights.md
├── notebooks/
│   └── diabetes_readmission_analysis.ipynb   the project notebook, Parts 1-5
└── sql/
    └── load_data.py
```

The notebook reads and writes through ../data, so it sits one folder below the root. The task also requires a README in the root (5.3).

5.3 README

The README describes the project and how to run it:

- **Project:** title, question (30-day readmission) and a short summary of the findings.

- **Data:** source (UCI Diabetes 130-US Hospitals, 1999-2008) and where to put diabetic_data.csv and IDS_mapping.csv (data/).
- **Setup:** Python 3 with numpy, pandas, matplotlib, seaborn, scipy and scikit-learn (the imports of the Setup cell).
- **Run:** open the notebook in notebooks/ and run all cells top to bottom.
- **Outputs:** the cleaned files in data/ and the Power BI files in data/powerbi/.
- **Folder structure and documentation:** the tree in 5.2 and links to the files in docs/.

5.4 Deliverables

Task deliverable	Covered by
Notebook: data cleaning and transformation steps	Notebook Part 2, Steps 1-17 (report section 2)
Notebook: EDA visualizations and summaries	Notebook Part 3, Steps 2-7 (report sections 3.1 and 3.2)
Notebook: insights and feature selection	Part 3, Step 8 (key insights); columns dropped in Part 2, Steps 3, 4 and 15; top features in Part 4, Step 7 (report 3.3 and Appendix)
Cleaned datasets	data/cleaned_readable.csv, data/cleaned.csv
Well-structured, commented code	Notebook Setup and Parts 1-5 (report section 4.2)
Git integration and GitHub repository with all project files	https://github.com/ManilBista/healthcare-readmission-analysis (report sections 5.1 and 5.2)
Clear and concise documentation	docs/cleaning_log.md, docs/insights.md, docs/dashboard_plan.md and this report
README with setup instructions	README.md in the repository root (report section 5.3)